Salon Dashboard App Tutorial

Oct 13, 2026 · @Divine Armour

Overview

You will build a private salon dashboard app for Android and iPhone, from one codebase, on a Chromebook, using only free tiers. Only you log in. Work through the parts in order; each one ends with something you can open on your phone.

What the app does

- **Today and this week:** today's bookings, the week's bookings, and client follow-ups due.
- **Revenue:** actual sales (paid) and potential sales (booked but unpaid), by day, week and month, shown as numbers and charts.
- **Appointment calendar:** book, reschedule, cancel and mark appointments done.
- **Client profiles:** contact details, visit history, notes and preferences.
- **Service menu:** name, category, price and duration. It can only be edited after entering a passcode.
- **Payments:** log cash, card, or bank transfer by hand, or send a client an online payment link.
- **Reminders:** push notifications to your own phone for upcoming appointments and follow-ups.

The tools and why

Layer	Tool	Why this one
App framework	Expo (React Native)	One JavaScript/TypeScript codebase for Android and iPhone; iPhone builds run in the cloud, so no Mac needed
Screens and navigation	Expo Router	Each file in the `app` folder becomes a screen
Database and login	Supabase	Free Postgres database and email login
Calendar	react-native-calendars	Ready-made month and week calendar views

Layer	Tool	Why this one
Charts	react-native-gifted-charts	Bar, line and pie charts that work in Expo
Reminders	expo-notifications	Schedules notifications on your phone itself, so no server or paid text service
Building and publishing	EAS Build and Submit	Cloud builds for the Play Store and App Store

How to read this tutorial

- Grey code blocks are things you type or paste. Lines starting with `$` are terminal commands; type them without the `$`.
- Every part ends with a **Check it works** step. Don't move on until it passes.
- File paths like `app/(tabs)/index.tsx` are relative to your project folder.

Accounts and costs

The software and services are all free. The only unavoidable costs are the two app store developer accounts, and you can put those off until Part 13.

Account	Cost	What the free tier gives you	When you need it
GitHub	Free	Unlimited private code repositories	Part 1
Expo	Free	30 cloud builds a month, up to 15 for iPhone (pricing)	Part 2
Supabase	Free	2 projects, 500 MB database, 50,000 monthly users (pricing)	Part 3
Google Play Console	US$25, one time	Publish to Android phones	Part 13
Apple Developer Program	US$99 a year	Publish to iPhones	Part 13

Free-tier catches to know about

- **Supabase pauses inactive projects.** A free project with little database activity over 7 days gets paused (Supabase docs). Using the app daily keeps it awake. If you take a break, open the Supabase dashboard once a week. A paused project can be restored from the dashboard, and your data is kept.
- **No automatic backups on Supabase Free.** Part 12 shows you how to export a backup yourself each month.
- **Expo cloud builds are queued.** On the free plan, builds can wait in line before starting. Build only when a part is finished, not after every small change.
- **You don't need the Apple account to test on an iPhone.** The free Expo Go app runs your app during development. You only need the paid account to publish to the App Store.

Part 1: Set up the Chromebook

By the end of this part you'll have a Linux terminal, Node.js, Git and a code editor on your Chromebook, plus Expo Go on your phone. Allow about an hour, mostly waiting for downloads.

1.1 Turn on Linux

1. Open **Settings → About ChromeOS → Developers → Linux development environment → Set up**. If you don't see it, check under **Settings → Advanced → Developers**.
2. Choose a username and set the disk size to at least **20 GB**. Node projects get large.
3. When it finishes, a **Terminal** window opens. Everything below is typed in that window.

If Linux is greyed out, your Chromebook may be managed by a school or workplace that blocks it. In that case use GitHub Codespaces, a browser-based coding environment with a free monthly allowance, and run the same commands there.

1.2 Update Linux and install basic tools

```
$ sudo apt update && sudo apt upgrade -y
$ sudo apt install -y git curl build-essential
```

1.3 Install Node.js with nvm

nvm (Node Version Manager) lets you install and switch Node versions without admin trouble.

```
$ curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.3/install.sh |
bash
```

Close the Terminal and open it again, then:

```
$ nvm install --lts
$ node -v
$ npm -v
```

Both commands should print a version number. If `nvm` isn't found, close and reopen the Terminal once more. Check the nvm page if a newer install script version is listed.

1.4 Set up Git

```
$ git config --global user.name "Your Name"
$ git config --global user.email "you@example.com"
```

Create a free GitHub account with the same email. You'll push your code there so it's backed up.

1.5 Install VS Code

1. In the Terminal, run `dpkg --print-architecture`. It prints `amd64` (most Chromebooks) or `arm64`.
2. Go to code.visualstudio.com/download and download the matching **.deb** file.
3. In the ChromeOS **Files** app, move the file into **Linux files**, double-click it and choose **Install**.
4. Open VS Code from the app launcher (under **Linux apps**). Install these extensions from the Extensions panel: **ES7+ React/Redux/React-Native snippets**, **Prettier**, and **Expo Tools**.

1.6 Get your phone ready

1. Install **Expo Go** from the Play Store or the App Store.
2. Create a free account at expo.dev and sign in to Expo Go with it.

Check it works

- [ ] `node -v` prints a version
- [ ] `git --version` prints a version

- [ ] VS Code opens
- [ ] Expo Go is installed and signed in on your phone

Part 2: Create the app and its screens

By the end of this part the app runs on your phone with five empty tabs: Home, Calendar, Clients, Sales and More.

2.1 Create the project

```
$ mkdir -p ~/projects && cd ~/projects
$ npx create-expo-app@latest salon-dashboard
$ cd salon-dashboard
$ npm run reset-project
```

`reset-project` moves the example screens into an `app-example` folder and gives you a blank `app` folder. Delete `app-example` once you've had a look.

2.2 Install the libraries for the whole app

Install everything now so later parts are just code. `npx expo install` picks versions that match your Expo version, so use it for anything Expo-related.

```
$ npx expo install @supabase/supabase-js @react-native-async-storage/async-storage react-native-url-polyfill expo-secure-store expo-crypto expo-notifications expo-device react-native-svg expo-linear-gradient @react-native-community/datetimepicker
$ npm install react-native-calendars react-native-gifted-charts date-fns
```

Library	Used for
@supabase/supabase-js	Talking to your database and login
async-storage	Keeps you logged in between app launches
expo-secure-store, expo-crypto	Storing the service-menu passcode safely
expo-notifications, expo-device	Reminder notifications
react-native-svg, expo-linear-gradient	Needed by the charts library
datetimepicker	Picking dates and times for bookings

Library	Used for
react-native-calendars	The calendar screen
react-native-gifted-charts	Revenue charts
date-fns	Date maths: start of week, add days, formatting

2.3 Folder layout

Create these folders and empty files now. Each part fills some of them in.

```
salon-dashboard/
  app/
    _layout.tsx           root: checks login, wraps every screen
    login.tsx             owner login
    (tabs)/
      _layout.tsx         the bottom tab bar
      index.tsx           Home dashboard
      calendar.tsx        appointment calendar
      clients.tsx         client list
      sales.tsx           sales and revenue
      more.tsx            services, settings, logout
    client/[id].tsx       one client's profile
    client/new.tsx        add a client
    appointment/[id].tsx  one appointment
    appointment/new.tsx   book an appointment
    services.tsx          service menu (passcode-locked)
    payment/new.tsx       record a payment or send a link
  lib/
    supabase.ts           database connection
    types.ts              shapes of your data
    money.ts              TT$ formatting
    passcode.ts           passcode lock
    notifications.ts      reminders
  components/
    Screen.tsx            shared page wrapper
    StatCard.tsx          number tiles on the dashboard
```

Files in `app/` become screens automatically. A folder in brackets, like `(tabs)`, groups screens without adding to the address. `[id]` means the screen takes an ID, for example `client/42`.

2.4 The tab bar

`app/(tabs)/_layout.tsx`:

```
import { Tabs } from 'expo-router';
import { Ionicons } from '@expo/vector-icons';

const PINK = '#C2185B';

export default function TabLayout() {
  return (
    <Tabs screenOptions={{ tabBarActiveTintColor: PINK, headerTitleAlign:
'center' }}>
      <Tabs.Screen name="index" options={{ title: 'Home',
        tabBarIcon: ({ color, size }) => <Ionicons name="home" color={color}
size={size} /> }} />
      <Tabs.Screen name="calendar" options={{ title: 'Calendar',
        tabBarIcon: ({ color, size }) => <Ionicons name="calendar" color=
{color} size={size} /> }} />
      <Tabs.Screen name="clients" options={{ title: 'Clients',
        tabBarIcon: ({ color, size }) => <Ionicons name="people" color=
{color} size={size} /> }} />
      <Tabs.Screen name="sales" options={{ title: 'Sales',
        tabBarIcon: ({ color, size }) => <Ionicons name="cash" color={color}
size={size} /> }} />
      <Tabs.Screen name="more" options={{ title: 'More',
        tabBarIcon: ({ color, size }) => <Ionicons name="menu" color={color}
size={size} /> }} />
    </Tabs>
  );
}
```

For now, put a placeholder in each tab file, for example `app/(tabs)/calendar.tsx`:

```
import { Text, View } from 'react-native';

export default function Calendar() {
  return <View style={{ flex: 1, alignItems: 'center', justifyContent:
'center' }}><Text>Calendar</Text></View>;
}
```

A temporary `app/_layout.tsx` (Part 4 replaces it):

```
import { Stack } from 'expo-router';

export default function RootLayout() {
  return <Stack screenOptions={{ headerShown: false }} />;
}
```

2.5 Shared pieces

`lib/money.ts` formats Trinidad & Tobago dollars:

```
export const TTD = (n: number) =>
  'TT$' + n.toLocaleString('en-TT', { minimumFractionDigits: 2,
maximumFractionDigits: 2 });
```

`components/Screen.tsx` gives every page the same padding and background:

```
import { ScrollView, StyleSheet, ViewStyle } from 'react-native';

export default function Screen({ children, style }: { children:
React.ReactNode; style?: ViewStyle }) {
  return <ScrollView contentContainerStyle={[styles.c, style]}>{children}
</ScrollView>;
}
const styles = StyleSheet.create({ c: { padding: 16, gap: 12,
backgroundColor: '#FFF7FA', flexGrow: 1 } });
```

2.6 Run it on your phone

```
$ npx expo start --tunnel
```

Use `--tunnel` on a Chromebook. Linux runs in its own small virtual network, so your phone usually can't reach it directly. The first time, Expo asks to install `@expo/ngrok`; say yes. Scan the QR code with the Expo Go app (Android) or the Camera app (iPhone).

Save your work to GitHub: create an empty private repository on github.com called `salon-dashboard`, then:

```
$ git add . && git commit -m "Part 2: app skeleton"
$ git remote add origin https://github.com/YOUR-USERNAME/salon-dashboard.git
$ git push -u origin main
```

GitHub will ask for a password: paste a **personal access token** from GitHub → Settings → Developer settings, not your account password.

Check it works

- [ ] The app opens in Expo Go and shows five tabs
- [ ] Changing the word "Calendar" in the code and saving updates the phone within a few seconds
- [ ] Your code is visible in your GitHub repository

Part 3: The database

By the end of this part you'll have four tables (services, clients, appointments and payments), security rules so only you can see the data, and the app connected to the database.

3.1 Create the Supabase project

1. Sign up at supabase.com with your GitHub account.
2. Click **New project**. Name it `salon-dashboard`, set a strong database password and save it in a password manager.
3. For **Region**, pick **East US (North Virginia)**. It's the closest to Trinidad, which keeps the app fast.
4. Wait about two minutes for it to finish setting up.

3.2 How the data fits together

- A **client** has many **appointments**.
- Each **appointment** is for one **service**. It copies the service's price at booking time, so changing the menu later doesn't rewrite old sales.
- A **payment** usually belongs to an appointment, but can also stand alone (for example, a tip or a product sold without a booking).
- **Actual sales** = payments marked paid. **Potential sales** = what's still owed on booked appointments.

3.3 Create the tables

In Supabase, open **SQL Editor → New query**, paste all of this, and click **Run**.

```
-- SERVICES: your menu
create table services (
  id uuid primary key default gen_random_uuid(),
  owner_id uuid not null default auth.uid() references auth.users(id),
  name text not null,
  category text not null default 'General',
  price numeric(10,2) not null check (price >= 0),
  duration_min int not null default 60 check (duration_min > 0),
  active boolean not null default true,
  created_at timestamptz not null default now()
);

-- CLIENTS: profiles and follow-ups
create table clients (
  id uuid primary key default gen_random_uuid(),
  owner_id uuid not null default auth.uid() references auth.users(id),
  full_name text not null,
  phone text,
  email text,
  birthday date,
  preferences text,          -- e.g. "prefers quiet, sensitive scalp"
  notes text,
  follow_up_date date,       -- when to check in with this client
  follow_up_note text,
  created_at timestamptz not null default now()
);

-- APPOINTMENTS: the calendar
create table appointments (
  id uuid primary key default gen_random_uuid(),
  owner_id uuid not null default auth.uid() references auth.users(id),
  client_id uuid not null references clients(id) on delete cascade,
  service_id uuid references services(id) on delete set null,
  service_name text not null,            -- copied at booking
  price numeric(10,2) not null,          -- copied at booking
  starts_at timestamptz not null,
  ends_at timestamptz not null,
  status text not null default 'booked'
    check (status in ('booked','completed','cancelled','no_show')),
  notes text,
  created_at timestamptz not null default now()
);
create index on appointments (starts_at);
```

```
-- PAYMENTS: manual and online
create table payments (
  id uuid primary key default gen_random_uuid(),
  owner_id uuid not null default auth.uid() references auth.users(id),
  appointment_id uuid references appointments(id) on delete set null,
  client_id uuid references clients(id) on delete set null,
  amount numeric(10,2) not null check (amount > 0),
  method text not null check (method in ('cash','card','transfer','online')),
  status text not null default 'paid'
    check (status in ('pending','paid','failed','refunded')),
  paid_at timestamptz,
  reference text,       -- receipt no. or bank ref
  payment_url text,     -- online payment link
  note text,
  created_at timestamptz not null default now()
);
create index on payments (paid_at);
```

3.4 Lock it down: row level security

Row level security (RLS) means the database refuses any row that isn't yours, even if someone gets hold of your app's key. Run this as a new query:

```
alter table services     enable row level security;
alter table clients      enable row level security;
alter table appointments enable row level security;
alter table payments     enable row level security;

create policy "owner only" on services     for all using (owner_id =
auth.uid()) with check (owner_id = auth.uid());
create policy "owner only" on clients      for all using (owner_id =
auth.uid()) with check (owner_id = auth.uid());
create policy "owner only" on appointments for all using (owner_id =
auth.uid()) with check (owner_id = auth.uid());
create policy "owner only" on payments     for all using (owner_id =
auth.uid()) with check (owner_id = auth.uid());
```

3.5 A balance view for potential sales

This view works out, for every appointment, how much has been paid and how much is still owed. `security_invoker` makes the view obey the RLS rules above.

```
create view appointment_balances with (security_invoker = true) as
select a.*,
       c.full_name as client_name,
       coalesce(sum(p.amount) filter (where p.status = 'paid'), 0) as paid,
       a.price - coalesce(sum(p.amount) filter (where p.status = 'paid'), 0)
as balance
from appointments a
join clients c on c.id = a.client_id
left join payments p on p.appointment_id = a.id
group by a.id, c.full_name;
```

3.6 Connect the app

1. In Supabase, open **Project Settings → API Keys** (on some projects it's **Settings → API**). Copy the **Project URL** and the **publishable key**. Older projects call it the **anon** key.
2. In your project folder, create a file named `.env`:

```
EXPO_PUBLIC_SUPABASE_URL=https://YOUR-PROJECT.supabase.co
EXPO_PUBLIC_SUPABASE_KEY=YOUR-PUBLISHABLE-OR-ANON-KEY
```

3. Open `.gitignore` and add a line `.env` so the keys never go to GitHub.

The publishable key is meant to be in apps; RLS is what protects your data. **Never** put the `service_role` or secret key in the app.

`lib/supabase.ts`:

```
import 'react-native-url-polyfill/auto';
import AsyncStorage from '@react-native-async-storage/async-storage';
import { createClient } from '@supabase/supabase-js';
import { AppState } from 'react-native';

export const supabase = createClient(
  process.env.EXPO_PUBLIC_SUPABASE_URL!,
  process.env.EXPO_PUBLIC_SUPABASE_KEY!,
  { auth: { storage: AsyncStorage, autoRefreshToken: true, persistSession:
true, detectSessionInUrl: false } }
);

// Refresh the login token only while the app is open
AppState.addEventListener('change', (state) => {
  if (state === 'active') supabase.auth.startAutoRefresh();
  else supabase.auth.stopAutoRefresh();
});
```

`lib/types.ts`:

```
export type Service = { id: string; name: string; category: string; price:
number; duration_min: number; active: boolean };
export type Client = { id: string; full_name: string; phone: string | null;
email: string | null; birthday: string | null;
  preferences: string | null; notes: string | null; follow_up_date: string |
null; follow_up_note: string | null };
export type AppointmentStatus = 'booked' | 'completed' | 'cancelled' |
'no_show';
export type Appointment = { id: string; client_id: string; service_id: string
| null; service_name: string; price: number;
  starts_at: string; ends_at: string; status: AppointmentStatus; notes:
string | null };
export type AppointmentBalance = Appointment & { client_name: string; paid:
number; balance: number };
export type Payment = { id: string; appointment_id: string | null; client_id:
string | null; amount: number;
  method: 'cash' | 'card' | 'transfer' | 'online'; status: 'pending' | 'paid'
| 'failed' | 'refunded';
  paid_at: string | null; reference: string | null; payment_url: string |
null; note: string | null };
```

Restart Expo after creating `.env` (press Ctrl+C, then `npx expo start --tunnel` again). It only reads the file at startup.

Check it works

- [ ] **Table Editor** in Supabase shows services, clients, appointments and payments
- [ ] Each table shows a green **RLS enabled** label
- [ ] `.env` is listed in `.gitignore`

Part 4: Owner login and passcode lock

By the end of this part the app opens on a login screen, stays logged in after that, and has a passcode system ready to lock the service menu.

4.1 Create your account and close sign-ups

Since only you use the app, you create your one account by hand and then switch off sign-ups so nobody else can make one.

1. In Supabase, open **Authentication → Users → Add user → Create new user**. Enter your email and a strong password, and tick **Auto Confirm User**.
2. Open **Authentication → Sign In / Providers** and turn **off** "Allow new users to sign up".

4.2 The root layout: show login or the app

Replace `app/_layout.tsx`. `Stack.Protected` shows a group of screens only when its `guard` is true, so logged-out users can only ever see `login`.

```
import { useEffect, useState } from 'react';
import { ActivityIndicator, View } from 'react-native';
import { Stack } from 'expo-router';
import type { Session } from '@supabase/supabase-js';
import { supabase } from '../lib/supabase';

export default function RootLayout() {
  const [session, setSession] = useState<Session | null>(null);
  const [ready, setReady] = useState(false);

  useEffect(() => {
    supabase.auth.getSession().then(({ data }) => { setSession(data.session);
setReady(true); });
```

```
    const { data: sub } = supabase.auth.onAuthStateChange((_event, s) =>
setSession(s));
    return () => sub.subscription.unsubscribe();
  }, []);

  if (!ready) return <View style={{ flex: 1, justifyContent: 'center' }}>
<ActivityIndicator /></View>;

  return (
    <Stack>
      <Stack.Protected guard={!!session}>
        <Stack.Screen name="(tabs)" options={{ headerShown: false }} />
        <Stack.Screen name="client/[id]" options={{ title: 'Client' }} />
        <Stack.Screen name="client/new" options={{ title: 'New client',
presentation: 'modal' }} />
        <Stack.Screen name="appointment/[id]" options={{ title: 'Appointment'
}} />
        <Stack.Screen name="appointment/new" options={{ title: 'Book
appointment', presentation: 'modal' }} />
        <Stack.Screen name="services" options={{ title: 'Service menu' }} />
        <Stack.Screen name="payment/new" options={{ title: 'Payment',
presentation: 'modal' }} />
      </Stack.Protected>
      <Stack.Protected guard={!session}>
        <Stack.Screen name="login" options={{ headerShown: false }} />
      </Stack.Protected>
    </Stack>
  );
}
```

4.3 The login screen

`app/login.tsx`:

```
import { useState } from 'react';
import { Alert, Pressable, StyleSheet, Text, TextInput, View } from 'react-
native';
import { supabase } from '../lib/supabase';

export default function Login() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [busy, setBusy] = useState(false);
```

```
  async function signIn() {
    setBusy(true);
    const { error } = await supabase.auth.signInWithPassword({ email:
email.trim(), password });
    setBusy(false);
    if (error) Alert.alert('Login failed', error.message);
    // On success, the root layout sees the new session and shows the tabs.
  }

  return (
    <View style={s.wrap}>
      <Text style={s.title}>Salon Dashboard</Text>
      <TextInput style={s.input} placeholder="Email" autoCapitalize="none"
keyboardType="email-address"
        value={email} onChangeText={setEmail} />
      <TextInput style={s.input} placeholder="Password" secureTextEntry
value={password} onChangeText={setPassword} />
      <Pressable style={[s.btn, busy && { opacity: 0.5 }]} disabled={busy}
onPress={signIn}>
        <Text style={s.btnText}>{busy ? 'Signing in…' : 'Sign in'}</Text>
      </Pressable>
    </View>
  );
}

const s = StyleSheet.create({
  wrap: { flex: 1, justifyContent: 'center', padding: 24, gap: 12,
backgroundColor: '#FFF7FA' },
  title: { fontSize: 28, fontWeight: '700', textAlign: 'center',
marginBottom: 24, color: '#C2185B' },
  input: { backgroundColor: 'white', borderRadius: 10, padding: 14,
borderWidth: 1, borderColor: '#eee' },
  btn: { backgroundColor: '#C2185B', borderRadius: 10, padding: 16,
alignItems: 'center' },
  btnText: { color: 'white', fontWeight: '600', fontSize: 16 },
});
```

4.4 The passcode helper

The passcode is stored on the phone in its secure storage (the Android Keystore or iPhone Keychain), and only as a scrambled hash, never as the digits themselves. If you forget it, your account password resets it.

`lib/passcode.ts`:

```
import * as SecureStore from 'expo-secure-store';
import * as Crypto from 'expo-crypto';
import { supabase } from './supabase';

const KEY = 'menu_passcode_hash';
const SALT = 'salon-dashboard-v1';
const hash = (code: string) =>
Crypto.digestStringAsync(Crypto.CryptoDigestAlgorithm.SHA256, SALT + code);

export async function hasPasscode() {
  return (await SecureStore.getItemAsync(KEY)) !== null;
}
export async function setPasscode(code: string) {
  if (!/^\d{4,6}$/.test(code)) throw new Error('Use 4 to 6 digits');
  await SecureStore.setItemAsync(KEY, await hash(code));
}
export async function checkPasscode(code: string) {
  return (await SecureStore.getItemAsync(KEY)) === (await hash(code));
}
// Forgot it? Prove it's you with your account password, then set a new one.
export async function resetWithPassword(password: string) {
  const { data } = await supabase.auth.getUser();
  const { error } = await supabase.auth.signInWithPassword({ email:
data.user!.email!, password });
  if (error) throw new Error('Wrong password');
  await SecureStore.deleteItemAsync(KEY);
}
```

4.5 A reusable passcode prompt

`components/PasscodePrompt.tsx` shows a pop-up asking for the passcode. On the first use it asks you to create one. After 5 wrong tries it locks for 60 seconds.

```
import { useEffect, useRef, useState } from 'react';
import { Alert, Modal, Pressable, StyleSheet, Text, TextInput, View } from
'react-native';
import { checkPasscode, hasPasscode, resetWithPassword, setPasscode } from
'../lib/passcode';

type Props = { visible: boolean; onSuccess: () => void; onCancel: () => void
};

export default function PasscodePrompt({ visible, onSuccess, onCancel }:
Props) {
  const [code, setCode] = useState('');
  const [creating, setCreating] = useState(false);
  const fails = useRef(0);
  const lockedUntil = useRef(0);

  useEffect(() => { if (visible) { setCode(''); hasPasscode().then((h) =>
setCreating(!h)); } }, [visible]);

  async function submit() {
    if (Date.now() < lockedUntil.current) return Alert.alert('Locked', 'Too
many tries. Wait a minute.');
    if (creating) {
      try { await setPasscode(code); onSuccess(); } catch (e: any) {
Alert.alert('Passcode', e.message); }
      return;
    }
    if (await checkPasscode(code)) { fails.current = 0; onSuccess(); return;
}
    fails.current += 1; setCode('');
    if (fails.current >= 5) { lockedUntil.current = Date.now() + 60_000;
fails.current = 0; }
    Alert.alert('Wrong passcode');
  }

  function forgot() {
    Alert.prompt?.('Reset passcode', 'Enter your account password', async
(pw) => {
      try { await resetWithPassword(pw); setCreating(true);
Alert.alert('Done', 'Now create a new passcode.'); }
      catch (e: any) { Alert.alert(e.message); }
    }, 'secure-text');
    // Alert.prompt is iPhone-only. On Android, log out and back in from the
```

```
More tab, then use "Change passcode".
  }

  return (
    <Modal visible={visible} transparent animationType="fade" onRequestClose=
{onCancel}>
      <View style={s.backdrop}>
        <View style={s.card}>
          <Text style={s.title}>{creating ? 'Create a 4–6 digit passcode' :
'Enter passcode'}</Text>
          <TextInput style={s.input} value={code} onChangeText={setCode}
keyboardType="number-pad"
            secureTextEntry maxLength={6} autoFocus onSubmitEditing={submit}
/>
          <Pressable style={s.btn} onPress={submit}><Text style=
{s.btnText}>OK</Text></Pressable>
          <Pressable onPress={onCancel}><Text style={s.link}>Cancel</Text>
</Pressable>
          {!creating && <Pressable onPress={forgot}><Text style=
{s.link}>Forgot passcode?</Text></Pressable>}
        </View>
      </View>
    </Modal>
  );
}

const s = StyleSheet.create({
  backdrop: { flex: 1, backgroundColor: '#0006', justifyContent: 'center',
padding: 32 },
  card: { backgroundColor: 'white', borderRadius: 16, padding: 20, gap: 12 },
  title: { fontSize: 18, fontWeight: '600', textAlign: 'center' },
  input: { borderWidth: 1, borderColor: '#ddd', borderRadius: 10, padding:
14, fontSize: 24, textAlign: 'center', letterSpacing: 8 },
  btn: { backgroundColor: '#C2185B', borderRadius: 10, padding: 14,
alignItems: 'center' },
  btnText: { color: 'white', fontWeight: '600' },
  link: { textAlign: 'center', color: '#C2185B', padding: 4 },
});
```

4.6 The More tab: services, passcode, log out

`app/(tabs)/more.tsx`:

```
import { useState } from 'react';
import { Alert, Pressable, Text } from 'react-native';
import { router } from 'expo-router';
import * as SecureStore from 'expo-secure-store';
import Screen from '../../components/Screen';
import PasscodePrompt from '../../components/PasscodePrompt';
import { supabase } from '../../lib/supabase';

export default function More() {
  const [changing, setChanging] = useState(false);
  const Row = ({ label, onPress }: { label: string; onPress: () => void }) =>
(
    <Pressable onPress={onPress} style={{ backgroundColor: 'white', padding:
16, borderRadius: 12 }}>
      <Text style={{ fontSize: 16 }}>{label}</Text>
    </Pressable>
  );
  return (
    <Screen>
      <Row label="Service menu" onPress={() => router.push('/services')} />
      <Row label="Change passcode" onPress={() => setChanging(true)} />
      <Row label="Log out" onPress={() => Alert.alert('Log out?', '', [
        { text: 'Cancel' },
        { text: 'Log out', style: 'destructive', onPress: async () => {
          await SecureStore.deleteItemAsync('menu_passcode_hash'); // you'll
set a new one after logging back in
          await supabase.auth.signOut();
        } },
      ])} />
      <PasscodePrompt visible={changing} onCancel={() => setChanging(false)}
        onSuccess={async () => {
          setChanging(false);
          await SecureStore.deleteItemAsync('menu_passcode_hash');
          Alert.alert('Old passcode accepted', 'Open the service menu to
create your new one.');
        }} />
    </Screen>
  );
}
```

Check it works

- [ ] The app opens on the login screen; a wrong password shows an error
- [ ] Your real login opens the tabs
- [ ] Closing and reopening the app keeps you logged in
- [ ] Log out returns you to the login screen

Part 5: Service menu (passcode-locked)

By the end of this part you can view your menu grouped by category, and add, edit or retire services only after entering the passcode. The lock resets every time you leave the screen.

Services are **retired, not deleted**. A retired service disappears from booking but stays attached to old appointments, so your sales history stays correct.

`app/services.tsx`:

```
import { useCallback, useState } from 'react';
import { Alert, Modal, Pressable, StyleSheet, Switch, Text, TextInput, View }
from 'react-native';
import { useFocusEffect, useNavigation } from 'expo-router';
import Screen from '../components/Screen';
import PasscodePrompt from '../components/PasscodePrompt';
import { supabase } from '../lib/supabase';
import { TTD } from '../lib/money';
import type { Service } from '../lib/types';

type Draft = { id?: string; name: string; category: string; price: string;
duration_min: string; active: boolean };
const EMPTY: Draft = { name: '', category: 'Hair', price: '', duration_min:
'60', active: true };

export default function Services() {
  const [items, setItems] = useState<Service[]>([]);
  const [unlocked, setUnlocked] = useState(false);
  const [asking, setAsking] = useState(false);
  const [draft, setDraft] = useState<Draft | null>(null);

  const load = useCallback(async () => {
    const { data, error } = await
supabase.from('services').select('*').order('category').order('name');
```

```
    if (error) Alert.alert('Could not load', error.message); else
setItems(data as Service[]);
  }, []);

  // Reload on open; re-lock on leaving
  useFocusEffect(useCallback(() => { load(); return () => setUnlocked(false);
}, [load]));

  async function save() {
    if (!draft) return;
    const price = Number(draft.price), duration = Number(draft.duration_min);
    if (!draft.name.trim()) return Alert.alert('Name is required');
    if (!(price >= 0)) return Alert.alert('Price must be a number');
    if (!(duration > 0)) return Alert.alert('Duration must be minutes, e.g.
45');
    const row = { name: draft.name.trim(), category: draft.category.trim() ||
'General', price,
      duration_min: duration, active: draft.active };
    const { error } = draft.id
      ? await supabase.from('services').update(row).eq('id', draft.id)
      : await supabase.from('services').insert(row);
    if (error) return Alert.alert('Save failed', error.message);
    setDraft(null); load();
  }

  // Group by category; hide retired ones unless unlocked
  const visible = items.filter((s) => unlocked || s.active);
  const groups = visible.reduce<Record<string, Service[]>>((acc, s) => {
    (acc[s.category] ??= []).push(s); return acc;
  }, {});

  return (
    <Screen>
      <Pressable style={[st.btn, unlocked && st.btnOutline]}
        onPress={() => (unlocked ? setDraft({ ...EMPTY }) :
setAsking(true))}>
        <Text style={[st.btnText, unlocked && { color: '#C2185B' }]}>
          {unlocked ? '+ Add service' : '🔒 Edit menu'}
        </Text>
      </Pressable>

      {Object.entries(groups).map(([cat, list]) => (
        <View key={cat} style={{ gap: 6 }}>
```

```
          <Text style={st.cat}>{cat}</Text>
          {list.map((s) => (
            <Pressable key={s.id} disabled={!unlocked}
              onPress={() => setDraft({ id: s.id, name: s.name, category:
s.category, price: String(s.price),
                duration_min: String(s.duration_min), active: s.active })}
              style={[st.row, !s.active && { opacity: 0.4 }]}>
              <View style={{ flex: 1 }}>
                <Text style={st.name}>{s.name}{!s.active ? ' (retired)' : ''}
</Text>
                <Text style={st.sub}>{s.duration_min} min</Text>
              </View>
              <Text style={st.price}>{TTD(Number(s.price))}</Text>
            </Pressable>
          ))}
        </View>
      ))}
      {visible.length === 0 && <Text style={st.sub}>No services yet. Tap Edit
menu to add your first.</Text>}

      <PasscodePrompt visible={asking} onCancel={() => setAsking(false)}
        onSuccess={() => { setAsking(false); setUnlocked(true); }} />

      <Modal visible={!!draft} animationType="slide"
presentationStyle="pageSheet" onRequestClose={() => setDraft(null)}>
        {draft && (
          <Screen>
            <Text style={st.cat}>{draft.id ? 'Edit service' : 'New service'}
</Text>
            <TextInput style={st.input} placeholder="Name (e.g. Silk press)"
value={draft.name}
              onChangeText={(name) => setDraft({ ...draft, name })} />
            <TextInput style={st.input} placeholder="Category (e.g. Hair,
Nails, Lashes)" value={draft.category}
              onChangeText={(category) => setDraft({ ...draft, category })}
/>
            <TextInput style={st.input} placeholder="Price in TT$"
keyboardType="decimal-pad" value={draft.price}
              onChangeText={(price) => setDraft({ ...draft, price })} />
            <TextInput style={st.input} placeholder="Duration in minutes"
keyboardType="number-pad"
              value={draft.duration_min} onChangeText={(duration_min) =>
setDraft({ ...draft, duration_min })} />
```

```
            <View style={st.switchRow}>
              <Text>Available for booking</Text>
              <Switch value={draft.active} onValueChange={(active) =>
setDraft({ ...draft, active })} />
            </View>
            <Pressable style={st.btn} onPress={save}><Text style=
{st.btnText}>Save</Text></Pressable>
            <Pressable onPress={() => setDraft(null)}><Text style=
{st.cancel}>Cancel</Text></Pressable>
          </Screen>
        )}
      </Modal>
    </Screen>
  );
}

const st = StyleSheet.create({
  btn: { backgroundColor: '#C2185B', borderRadius: 10, padding: 14,
alignItems: 'center' },
  btnOutline: { backgroundColor: 'white', borderWidth: 1, borderColor:
'#C2185B' },
  btnText: { color: 'white', fontWeight: '600' },
  cat: { fontSize: 18, fontWeight: '700', marginTop: 8 },
  row: { flexDirection: 'row', alignItems: 'center', backgroundColor:
'white', padding: 14, borderRadius: 12 },
  name: { fontSize: 16, fontWeight: '500' },
  sub: { color: '#777' },
  price: { fontWeight: '600' },
  input: { backgroundColor: 'white', borderRadius: 10, padding: 14,
borderWidth: 1, borderColor: '#eee' },
  switchRow: { flexDirection: 'row', justifyContent: 'space-between',
alignItems: 'center', padding: 4 },
  cancel: { textAlign: 'center', color: '#C2185B', padding: 8 },
});
```

How the lock works: `unlocked` lives only in this screen's memory. `useFocusEffect` sets it back to `false` whenever you navigate away, so the next visit asks again. Rows can't be tapped while locked (`disabled={!unlocked}`).

Check it works

- [ ] Tapping **Edit menu** the first time asks you to create a passcode

- [ ] Add 3–4 real services across two categories; they appear grouped
- [ ] Leave the screen and come back: it's locked again and rows can't be tapped
- [ ] Turning off "Available for booking" hides the service until you unlock

Part 6: Client profiles

By the end of this part you can search clients, add and edit them, and open a profile showing their details, preferences, follow-up, visit history, total spent and anything still owed.

6.1 Client list with search

`app/(tabs)/clients.tsx`:

```
import { useCallback, useState } from 'react';
import { FlatList, Pressable, StyleSheet, Text, TextInput, View } from
'react-native';
import { router, useFocusEffect } from 'expo-router';
import { supabase } from '../../lib/supabase';
import type { Client } from '../../lib/types';

export default function Clients() {
  const [list, setList] = useState<Client[]>([]);
  const [q, setQ] = useState('');

  const load = useCallback(async (term: string) => {
    let query =
supabase.from('clients').select('*').order('full_name').limit(200);
    if (term.trim()) query =
query.or(`full_name.ilike.%${term}%,phone.ilike.%${term}%`);
    const { data } = await query;
    setList((data as Client[]) ?? []);
  }, []);

  useFocusEffect(useCallback(() => { load(q); }, [load, q]));

  return (
    <View style={{ flex: 1, backgroundColor: '#FFF7FA' }}>
      <View style={s.top}>
        <TextInput style={s.search} placeholder="Search name or phone" value=
{q} onChangeText={setQ} />
```

```
        <Pressable style={s.add} onPress={() => router.push('/client/new')}>
<Text style={s.addText}>+</Text></Pressable>
      </View>
      <FlatList data={list} keyExtractor={(c) => c.id} contentContainerStyle=
{{ padding: 16, gap: 8 }}
        ListEmptyComponent={<Text style={{ color: '#777' }}>No clients yet.
</Text>}
        renderItem={({ item }) => (
          <Pressable style={s.row} onPress={() =>
router.push(`/client/${item.id}`)}>
            <Text style={s.name}>{item.full_name}</Text>
            <Text style={s.sub}>{item.phone ?? 'No phone'}</Text>
          </Pressable>
        )} />
    </View>
  );
}

const s = StyleSheet.create({
  top: { flexDirection: 'row', gap: 8, padding: 16, paddingBottom: 0 },
  search: { flex: 1, backgroundColor: 'white', borderRadius: 10, padding: 12,
borderWidth: 1, borderColor: '#eee' },
  add: { width: 48, borderRadius: 10, backgroundColor: '#C2185B', alignItems:
'center', justifyContent: 'center' },
  addText: { color: 'white', fontSize: 24 },
  row: { backgroundColor: 'white', padding: 14, borderRadius: 12 },
  name: { fontSize: 16, fontWeight: '500' },
  sub: { color: '#777', marginTop: 2 },
});
```

6.2 Add or edit a client

One screen does both. Open it as `/client/new` to add, or `/client/new?id=…` to edit. The follow-up buttons match how salon clients usually rebook: every 2, 4 or 6 weeks.

`app/client/new.tsx`:

```
import { useEffect, useState } from 'react';
import { Alert, Pressable, StyleSheet, Text, TextInput, View } from 'react-
native';
import { router, useLocalSearchParams } from 'expo-router';
import { addWeeks, format } from 'date-fns';
import Screen from '../../components/Screen';
```

```
import { supabase } from '../../lib/supabase';

const blank = { full_name: '', phone: '', email: '', birthday: '',
preferences: '', notes: '',
  follow_up_date: '', follow_up_note: '' };

export default function ClientForm() {
  const { id } = useLocalSearchParams<{ id?: string }>();
  const [f, setF] = useState(blank);

  useEffect(() => {
    if (!id) return;
    supabase.from('clients').select('*').eq('id', id).single().then(({ data
}) => {
      if (data) setF(Object.fromEntries(Object.keys(blank).map((k) => [k,
(data as any)[k] ?? ''])) as typeof blank);
    });
  }, [id]);

  const set = (k: keyof typeof blank) => (v: string) => setF({ ...f, [k]: v
});
  const followIn = (w: number) => setF({ ...f, follow_up_date:
format(addWeeks(new Date(), w), 'yyyy-MM-dd') });

  async function save() {
    if (!f.full_name.trim()) return Alert.alert('Name is required');
    if (f.birthday && !/^\d{4}-\d{2}-\d{2}$/.test(f.birthday)) return
Alert.alert('Birthday must look like 1990-05-21');
    // Empty strings become null so the database stores "nothing" rather than
""
    const row = Object.fromEntries(Object.entries(f).map(([k, v]) => [k,
v.trim() || null]));
    const { error } = id
      ? await supabase.from('clients').update(row).eq('id', id)
      : await supabase.from('clients').insert(row);
    if (error) return Alert.alert('Save failed', error.message);
    router.back();
  }

  return (
    <Screen>
      <TextInput style={s.in} placeholder="Full name *" value={f.full_name}
onChangeText={set('full_name')} />
```

```
      <TextInput style={s.in} placeholder="Phone (e.g. 868 555 1234)"
keyboardType="phone-pad" value={f.phone} onChangeText={set('phone')} />
      <TextInput style={s.in} placeholder="Email" autoCapitalize="none"
keyboardType="email-address" value={f.email} onChangeText={set('email')} />
      <TextInput style={s.in} placeholder="Birthday (YYYY-MM-DD)" value=
{f.birthday} onChangeText={set('birthday')} />
      <TextInput style={[s.in, s.multi]} multiline placeholder="Preferences
(products, allergies to products, style)" value={f.preferences} onChangeText=
{set('preferences')} />
      <TextInput style={[s.in, s.multi]} multiline placeholder="Notes" value=
{f.notes} onChangeText={set('notes')} />

      <Text style={s.label}>Follow up {f.follow_up_date ? `on
${f.follow_up_date}` : '(none)'}</Text>
      <View style={s.chips}>
        {[2, 4, 6].map((w) => (
          <Pressable key={w} style={s.chip} onPress={() => followIn(w)}>
<Text>In {w} wks</Text></Pressable>
        ))}
        <Pressable style={s.chip} onPress={() => setF({ ...f, follow_up_date:
'' })}><Text>Clear</Text></Pressable>
      </View>
      <TextInput style={s.in} placeholder="Follow-up note (e.g. check colour
fade)" value={f.follow_up_note} onChangeText={set('follow_up_note')} />

      <Pressable style={s.btn} onPress={save}><Text style={s.btnText}>Save
client</Text></Pressable>
    </Screen>
  );
}

const s = StyleSheet.create({
  in: { backgroundColor: 'white', borderRadius: 10, padding: 14, borderWidth:
1, borderColor: '#eee' },
  multi: { minHeight: 80, textAlignVertical: 'top' },
  label: { fontWeight: '600', marginTop: 4 },
  chips: { flexDirection: 'row', gap: 8, flexWrap: 'wrap' },
  chip: { backgroundColor: 'white', borderWidth: 1, borderColor: '#C2185B',
borderRadius: 20, paddingHorizontal: 12, paddingVertical: 6 },
  btn: { backgroundColor: '#C2185B', borderRadius: 10, padding: 16,
alignItems: 'center', marginTop: 8 },
  btnText: { color: 'white', fontWeight: '600' },
});
```

6.3 The client profile

The profile pulls the client and their appointments from the `appointment_balances` view, so it can show total spent and what's owed without extra maths. The call and WhatsApp buttons open your phone's own apps; nothing is sent automatically.

`app/client/[id].tsx`:

```
import { useCallback, useState } from 'react';
import { Alert, Linking, Pressable, StyleSheet, Text, View } from 'react-
native';
import { router, useFocusEffect, useLocalSearchParams } from 'expo-router';
import { format } from 'date-fns';
import Screen from '../../components/Screen';
import { supabase } from '../../lib/supabase';
import { TTD } from '../../lib/money';
import type { AppointmentBalance, Client } from '../../lib/types';

// "868 555 1234" -> "18685551234" for WhatsApp links
const waNumber = (p: string) => { const d = p.replace(/\D/g, ''); return
d.length === 7 ? '1868' + d : d.length === 10 ? '1' + d : d; };

export default function ClientProfile() {
  const { id } = useLocalSearchParams<{ id: string }>();
  const [c, setC] = useState<Client | null>(null);
  const [visits, setVisits] = useState<AppointmentBalance[]>([]);

  useFocusEffect(useCallback(() => {
    supabase.from('clients').select('*').eq('id', id).single().then(({ data
}) => setC(data as Client));
    supabase.from('appointment_balances').select('*').eq('client_id',
id).order('starts_at', { ascending: false })
      .then(({ data }) => setVisits((data as AppointmentBalance[]) ?? []));
  }, [id]));

  if (!c) return null;
  const done = visits.filter((v) => v.status === 'completed');
  const spent = visits.reduce((t, v) => t + Number(v.paid), 0);
  const owed = visits.filter((v) => v.status !== 'cancelled').reduce((t, v)
=> t + Math.max(0, Number(v.balance)), 0);

  async function remove() {
    Alert.alert('Delete client?', 'This also deletes their appointments.', [
```

```
      { text: 'Cancel' },
      { text: 'Delete', style: 'destructive', onPress: async () => {
        await supabase.from('clients').delete().eq('id', id); router.back();
} },
    ]);
  }

  return (
    <Screen>
      <Text style={s.name}>{c.full_name}</Text>
      <View style={s.actions}>
        {c.phone && <Pressable style={s.act} onPress={() =>
Linking.openURL(`tel:${c.phone}`)}><Text>Call</Text></Pressable>}
        {c.phone && <Pressable style={s.act} onPress={() =>
Linking.openURL(`https://wa.me/${waNumber(c.phone!)}`)}><Text>WhatsApp</Text>
</Pressable>}
        <Pressable style={s.act} onPress={() => router.push(`/client/new?
id=${id}`)}><Text>Edit</Text></Pressable>
      </View>

      <View style={s.stats}>
        <Stat label="Visits" value={String(done.length)} />
        <Stat label="Spent" value={TTD(spent)} />
        <Stat label="Owes" value={TTD(owed)} />
      </View>

      {c.follow_up_date && (
        <View style={s.card}><Text style={s.h}>Follow up {c.follow_up_date}
</Text><Text>{c.follow_up_note}</Text></View>
      )}
      {c.preferences && <View style={s.card}><Text style=
{s.h}>Preferences</Text><Text>{c.preferences}</Text></View>}
      {c.notes && <View style={s.card}><Text style={s.h}>Notes</Text><Text>
{c.notes}</Text></View>}

      <Pressable style={s.btn} onPress={() => router.push(`/appointment/new?
clientId=${id}`)}>
        <Text style={s.btnText}>Book appointment</Text>
      </Pressable>

      <Text style={s.h}>History</Text>
      {visits.map((v) => (
        <Pressable key={v.id} style={s.card} onPress={() =>
```

```
router.push(`/appointment/${v.id}`)}>
          <Text style={{ fontWeight: '500' }}>{v.service_name} ·
{TTD(Number(v.price))}</Text>
          <Text style={{ color: '#777' }}>{format(new Date(v.starts_at), 'EEE
d MMM yyyy, h:mm a')} · {v.status}
            {Number(v.balance) > 0 && v.status !== 'cancelled' ? ` · owes
${TTD(Number(v.balance))}` : ''}</Text>
        </Pressable>
      ))}
      <Pressable onPress={remove}><Text style={{ color: '#B00020', textAlign:
'center', padding: 12 }}>Delete client</Text></Pressable>
    </Screen>
  );
}

const Stat = ({ label, value }: { label: string; value: string }) => (
  <View style={s.stat}><Text style={{ color: '#777' }}>{label}</Text><Text
style={{ fontWeight: '700', fontSize: 16 }}>{value}</Text></View>
);

const s = StyleSheet.create({
  name: { fontSize: 24, fontWeight: '700' },
  actions: { flexDirection: 'row', gap: 8 },
  act: { backgroundColor: 'white', borderRadius: 20, paddingHorizontal: 14,
paddingVertical: 8, borderWidth: 1, borderColor: '#eee' },
  stats: { flexDirection: 'row', gap: 8 },
  stat: { flex: 1, backgroundColor: 'white', borderRadius: 12, padding: 12,
gap: 4 },
  card: { backgroundColor: 'white', borderRadius: 12, padding: 14, gap: 4 },
  h: { fontWeight: '700', fontSize: 16 },
  btn: { backgroundColor: '#C2185B', borderRadius: 10, padding: 14,
alignItems: 'center' },
  btnText: { color: 'white', fontWeight: '600' },
});
```

Check it works

- [ ] Add three clients; search finds them by part of a name or phone number
- [ ] Tapping "In 4 wks" fills a follow-up date four weeks from today
- [ ] Edit changes a client and the profile updates when you go back
- [ ] WhatsApp opens a chat with the right number

Part 7: Appointment calendar

By the end of this part you'll have a month calendar with dots on busy days, the day's bookings listed underneath, and a way to book, reschedule, cancel, and mark appointments done or no-show. The app warns you if a new booking overlaps another.

7.1 A date-and-time picker that works on both phones

Android and iPhone show date pickers differently. This component hides that difference.

`components/DateTimeField.tsx`:

```
import { Platform, Pressable, Text, View } from 'react-native';
import DateTimePicker, { DateTimePickerAndroid } from '@react-native-
community/datetimepicker';
import { format } from 'date-fns';

type Props = { value: Date; onChange: (d: Date) => void };

export default function DateTimeField({ value, onChange }: Props) {
  if (Platform.OS === 'ios') {
    return <DateTimePicker value={value} mode="datetime" minuteInterval={15}
      onChange={(_, d) => d && onChange(d)} />;
  }
  // Android: open the date dialog, then the time dialog
  const open = () => DateTimePickerAndroid.open({
    value, mode: 'date',
    onChange: (e, d) => {
      if (e.type !== 'set' || !d) return;
      DateTimePickerAndroid.open({
        value: d, mode: 'time', is24Hour: false,
        onChange: (e2, t) => { if (e2.type === 'set' && t) onChange(t); },
      });
    },
  });
  return (
    <Pressable onPress={open} style={{ backgroundColor: 'white', padding: 14,
borderRadius: 10, borderWidth: 1, borderColor: '#eee' }}>
      <Text>{format(value, 'EEE d MMM yyyy, h:mm a')}</Text>
    </Pressable>
  );
}
```

7.2 The calendar tab

`app/(tabs)/calendar.tsx` loads one month at a time. Dot colours: pink = booked, green = completed, grey = cancelled or no-show.

```
import { useCallback, useMemo, useState } from 'react';
import { FlatList, Pressable, StyleSheet, Text, View } from 'react-native';
import { Calendar } from 'react-native-calendars';
import { router, useFocusEffect } from 'expo-router';
import { endOfMonth, format, startOfMonth } from 'date-fns';
import { supabase } from '../../lib/supabase';
import { TTD } from '../../lib/money';
import type { AppointmentBalance } from '../../lib/types';

const COLOR = { booked: '#C2185B', completed: '#2E7D32', cancelled:
'#9E9E9E', no_show: '#9E9E9E' } as const;
const dayKey = (iso: string) => format(new Date(iso), 'yyyy-MM-dd');

export default function CalendarTab() {
  const [month, setMonth] = useState(new Date());
  const [selected, setSelected] = useState(format(new Date(), 'yyyy-MM-dd'));
  const [appts, setAppts] = useState<AppointmentBalance[]>([]);

  const load = useCallback(async () => {
    const { data } = await supabase.from('appointment_balances').select('*')
      .gte('starts_at', startOfMonth(month).toISOString())
      .lte('starts_at', endOfMonth(month).toISOString())
      .order('starts_at');
    setAppts((data as AppointmentBalance[]) ?? []);
  }, [month]);

  useFocusEffect(useCallback(() => { load(); }, [load]));

  const marked = useMemo(() => {
    const m: Record<string, any> = {};
    for (const a of appts) {
      const k = dayKey(a.starts_at);
      m[k] ??= { dots: [] };
      if (m[k].dots.length < 3) m[k].dots.push({ key: a.id, color:
COLOR[a.status] });
    }
    m[selected] = { ...(m[selected] ?? { dots: [] }), selected: true,
selectedColor: '#F8BBD0' };
```

```
    return m;
  }, [appts, selected]);

  const today = appts.filter((a) => dayKey(a.starts_at) === selected);

  return (
    <View style={{ flex: 1, backgroundColor: '#FFF7FA' }}>
      <Calendar markingType="multi-dot" markedDates={marked}
        onDayPress={(d) => setSelected(d.dateString)}
        onMonthChange={(d) => setMonth(new Date(d.year, d.month - 1, 1))}
        theme={{ todayTextColor: '#C2185B', arrowColor: '#C2185B' }} />
      <FlatList data={today} keyExtractor={(a) => a.id}
contentContainerStyle={{ padding: 16, gap: 8 }}
        ListHeaderComponent={<Text style={s.h}>{format(new Date(selected +
'T12:00'), 'EEEE d MMMM')}</Text>}
        ListEmptyComponent={<Text style={{ color: '#777' }}>No bookings.
</Text>}
        renderItem={({ item: a }) => (
          <Pressable style={[s.row, { borderLeftColor: COLOR[a.status] }]}
onPress={() => router.push(`/appointment/${a.id}`)}>
            <Text style={s.time}>{format(new Date(a.starts_at), 'h:mm a')}
</Text>
            <View style={{ flex: 1 }}>
              <Text style={{ fontWeight: '600' }}>{a.client_name}</Text>
              <Text style={{ color: '#777' }}>{a.service_name} ·
{TTD(Number(a.price))}</Text>
            </View>
          </Pressable>
        )} />
      <Pressable style={s.fab} onPress={() => router.push(`/appointment/new?
date=${selected}`)}>
        <Text style={{ color: 'white', fontSize: 28 }}>+</Text>
      </Pressable>
    </View>
  );
}

const s = StyleSheet.create({
  h: { fontWeight: '700', fontSize: 16, marginBottom: 4 },
  row: { flexDirection: 'row', gap: 12, backgroundColor: 'white', padding:
12, borderRadius: 12, borderLeftWidth: 4 },
  time: { width: 70, fontWeight: '600' },
  fab: { position: 'absolute', right: 20, bottom: 20, width: 56, height: 56,
```

```
borderRadius: 28,
    backgroundColor: '#C2185B', alignItems: 'center', justifyContent:
'center', elevation: 4 },
});
```

7.3 Book an appointment

The booking screen copies the service's name and price onto the appointment, works out the end time from the service's duration, and checks for overlaps before saving.

`app/appointment/new.tsx`:

```
import { useEffect, useState } from 'react';
import { Alert, Pressable, StyleSheet, Text, TextInput, View } from 'react-
native';
import { router, useLocalSearchParams } from 'expo-router';
import { addMinutes, setHours } from 'date-fns';
import Screen from '../../components/Screen';
import DateTimeField from '../../components/DateTimeField';
import { supabase } from '../../lib/supabase';
import { TTD } from '../../lib/money';
import { scheduleAppointmentReminder } from '../../lib/notifications';
import type { Client, Service } from '../../lib/types';

export default function BookAppointment() {
  const { clientId, date } = useLocalSearchParams<{ clientId?: string; date?:
string }>();
  const [clients, setClients] = useState<Client[]>([]);
  const [services, setServices] = useState<Service[]>([]);
  const [clientQ, setClientQ] = useState('');
  const [client, setClient] = useState<Client | null>(null);
  const [service, setService] = useState<Service | null>(null);
  const [start, setStart] = useState(() => setHours(date ? new Date(date +
'T00:00') : new Date(), 10));
  const [price, setPrice] = useState('');
  const [notes, setNotes] = useState('');

  useEffect(() => {
    supabase.from('services').select('*').eq('active',
true).order('name').then(({ data }) => setServices((data as Service[]) ??
[]));
    supabase.from('clients').select('*').order('full_name').then(({ data })
=> {
```

```
      const list = (data as Client[]) ?? []; setClients(list);
      if (clientId) setClient(list.find((c) => c.id === clientId) ?? null);
    });
  }, [clientId]);

  async function save() {
    if (!client || !service) return Alert.alert('Pick a client and a
service');
    const end = addMinutes(start, service.duration_min);
    // Overlap check: any booked appointment that starts before this ends and
ends after this starts
    const { data: clash } = await supabase.from('appointments').select('id,
service_name, starts_at')
      .eq('status', 'booked').lt('starts_at',
end.toISOString()).gt('ends_at', start.toISOString());
    const go = async () => {
      const { data, error } = await supabase.from('appointments').insert({
        client_id: client.id, service_id: service.id, service_name:
service.name,
        price: price ? Number(price) : service.price,
        starts_at: start.toISOString(), ends_at: end.toISOString(), notes:
notes || null,
      }).select().single();
      if (error) return Alert.alert('Save failed', error.message);
      await scheduleAppointmentReminder(data.id, client.full_name,
service.name, start);
      router.back();
    };
    if (clash && clash.length) {
      Alert.alert('Overlaps another booking', 'Book anyway?', [{ text:
'Cancel' }, { text: 'Book anyway', onPress: go }]);
    } else go();
  }

  const matches = clientQ ? clients.filter((c) =>
c.full_name.toLowerCase().includes(clientQ.toLowerCase())).slice(0, 5) : [];

  return (
    <Screen>
      <Text style={s.h}>Client</Text>
      {client ? (
        <Pressable style={s.picked} onPress={() => setClient(null)}><Text>
{client.full_name}  ×</Text></Pressable>
```

```
      ) : (
        <>
          <TextInput style={s.in} placeholder="Type a client's name" value=
{clientQ} onChangeText={setClientQ} />
          {matches.map((c) => (
            <Pressable key={c.id} style={s.opt} onPress={() => {
setClient(c); setClientQ(''); }}><Text>{c.full_name}</Text></Pressable>
          ))}
          <Pressable onPress={() => router.push('/client/new')}><Text style=
{s.link}>+ New client</Text></Pressable>
        </>
      )}

      <Text style={s.h}>Service</Text>
      <View style={s.wrap}>
        {services.map((sv) => (
          <Pressable key={sv.id} onPress={() => { setService(sv);
setPrice(''); }}
            style={[s.chip, service?.id === sv.id && s.chipOn]}>
            <Text style={service?.id === sv.id ? { color: 'white' } :
undefined}>{sv.name} · {TTD(Number(sv.price))}</Text>
          </Pressable>
        ))}
      </View>

      <Text style={s.h}>Date and time</Text>
      <DateTimeField value={start} onChange={setStart} />

      <TextInput style={s.in} keyboardType="decimal-pad" value={price}
onChangeText={setPrice}
        placeholder={service ? `Price (default
${TTD(Number(service.price))})` : 'Price'} />
      <TextInput style={s.in} placeholder="Notes" value={notes} onChangeText=
{setNotes} />
      <Pressable style={s.btn} onPress={save}><Text style=
{s.btnText}>Book</Text></Pressable>
    </Screen>
  );
}

const s = StyleSheet.create({
  h: { fontWeight: '700', fontSize: 16, marginTop: 4 },
  in: { backgroundColor: 'white', borderRadius: 10, padding: 14, borderWidth:
```

```
1, borderColor: '#eee' },
  opt: { backgroundColor: 'white', padding: 12, borderRadius: 8 },
  picked: { backgroundColor: '#F8BBD0', padding: 12, borderRadius: 10 },
  link: { color: '#C2185B', padding: 4 },
  wrap: { flexDirection: 'row', flexWrap: 'wrap', gap: 8 },
  chip: { backgroundColor: 'white', borderWidth: 1, borderColor: '#C2185B',
borderRadius: 20, paddingHorizontal: 12, paddingVertical: 8 },
  chipOn: { backgroundColor: '#C2185B' },
  btn: { backgroundColor: '#C2185B', borderRadius: 10, padding: 16,
alignItems: 'center', marginTop: 8 },
  btnText: { color: 'white', fontWeight: '600' },
});
```

`scheduleAppointmentReminder` comes from Part 11. Until then, create `lib/notifications.ts` with a placeholder so the app runs:

```
export async function scheduleAppointmentReminder(..._args: any[]) {}
export async function cancelAppointmentReminder(..._args: any[]) {}
```

7.4 One appointment: status, reschedule, payment

`app/appointment/[id].tsx`:

```
import { useCallback, useState } from 'react';
import { Alert, Pressable, StyleSheet, Text, View } from 'react-native';
import { router, useFocusEffect, useLocalSearchParams } from 'expo-router';
import { differenceInMinutes, addMinutes, format } from 'date-fns';
import Screen from '../../components/Screen';
import DateTimeField from '../../components/DateTimeField';
import { supabase } from '../../lib/supabase';
import { TTD } from '../../lib/money';
import { cancelAppointmentReminder, scheduleAppointmentReminder } from
'../../lib/notifications';
import type { AppointmentBalance, AppointmentStatus, Payment } from
'../../lib/types';

export default function AppointmentDetail() {
  const { id } = useLocalSearchParams<{ id: string }>();
  const [a, setA] = useState<AppointmentBalance | null>(null);
  const [pays, setPays] = useState<Payment[]>([]);

  const load = useCallback(async () => {
```

```
    const { data } = await
supabase.from('appointment_balances').select('*').eq('id', id).single();
    setA(data as AppointmentBalance);
    const { data: p } = await
supabase.from('payments').select('*').eq('appointment_id',
id).order('created_at');
    setPays((p as Payment[]) ?? []);
  }, [id]);
  useFocusEffect(useCallback(() => { load(); }, [load]));

  if (!a) return null;

  async function setStatus(status: AppointmentStatus) {
    await supabase.from('appointments').update({ status }).eq('id', id);
    if (status !== 'booked') await cancelAppointmentReminder(id);
    load();
  }
  async function reschedule(newStart: Date) {
    const len = differenceInMinutes(new Date(a!.ends_at), new
Date(a!.starts_at));
    await supabase.from('appointments').update({ starts_at:
newStart.toISOString(),
      ends_at: addMinutes(newStart, len).toISOString() }).eq('id', id);
    await cancelAppointmentReminder(id);
    await scheduleAppointmentReminder(id, a!.client_name, a!.service_name,
newStart);
    load();
  }

  return (
    <Screen>
      <Text style={s.title}>{a.client_name}</Text>
      <Text style={s.sub}>{a.service_name} · {TTD(Number(a.price))} ·
{a.status}</Text>
      {a.status === 'booked' && <DateTimeField value={new Date(a.starts_at)}
onChange={reschedule} />}
      {a.status !== 'booked' && <Text>{format(new Date(a.starts_at), 'EEE d
MMM yyyy, h:mm a')}</Text>}

      <View style={s.card}>
        <Text>Paid: {TTD(Number(a.paid))}</Text>
        <Text style={{ fontWeight: '700' }}>Balance: {TTD(Math.max(0,
Number(a.balance)))}</Text>
```

```
        {pays.map((p) => (
          <Text key={p.id} style={s.sub}>{p.method} · {TTD(Number(p.amount))}
· {p.status}</Text>
        ))}
      </View>

      {Number(a.balance) > 0 && a.status !== 'cancelled' && (
        <Pressable style={s.btn} onPress={() => router.push(`/payment/new?
appointmentId=${id}`)}>
          <Text style={s.btnText}>Take payment</Text>
        </Pressable>
      )}

      {a.status === 'booked' && (
        <View style={s.row}>
          <Pressable style={[s.small, { backgroundColor: '#2E7D32' }]}
onPress={() => setStatus('completed')}><Text style={s.btnText}>Done</Text>
</Pressable>
          <Pressable style={[s.small, { backgroundColor: '#757575' }]}
onPress={() => setStatus('no_show')}><Text style={s.btnText}>No-show</Text>
</Pressable>
          <Pressable style={[s.small, { backgroundColor: '#B00020' }]}
onPress={() =>
            Alert.alert('Cancel appointment?', '', [{ text: 'Keep' }, { text:
'Cancel it', style: 'destructive', onPress: () => setStatus('cancelled')
}])}>
            <Text style={s.btnText}>Cancel</Text></Pressable>
        </View>
      )}
      {a.status !== 'booked' && (
        <Pressable onPress={() => setStatus('booked')}><Text style={{ color:
'#C2185B', textAlign: 'center' }}>Undo (back to booked)</Text></Pressable>
      )}
      <Pressable onPress={() => router.push(`/client/${a.client_id}`)}><Text
style={{ color: '#C2185B', textAlign: 'center' }}>View client</Text>
</Pressable>
    </Screen>
  );
}

const s = StyleSheet.create({
  title: { fontSize: 22, fontWeight: '700' },
  sub: { color: '#777' },
```

```
  card: { backgroundColor: 'white', borderRadius: 12, padding: 14, gap: 4 },
  btn: { backgroundColor: '#C2185B', borderRadius: 10, padding: 14,
alignItems: 'center' },
  btnText: { color: 'white', fontWeight: '600' },
  row: { flexDirection: 'row', gap: 8 },
  small: { flex: 1, borderRadius: 10, padding: 12, alignItems: 'center' },
});
```

Check it works

- [ ] Book three appointments on different days; dots appear on those days
- [ ] Booking a time that overlaps another shows the warning
- [ ] Rescheduling moves the appointment to the new day
- [ ] Marking one Done turns its dot green; Cancel turns it grey

Part 8: Payments

By the end of this part you can record cash, card and bank-transfer payments in seconds. If a client pays online, you can paste their payment link, share it through WhatsApp, and mark it paid once the money arrives.

8.1 Record a payment

`app/payment/new.tsx` opens from an appointment with the balance already filled in, or on its own for walk-in sales and tips.

```
import { useEffect, useState } from 'react';
import { Alert, Pressable, Share, StyleSheet, Text, TextInput, View } from
'react-native';
import { router, useLocalSearchParams } from 'expo-router';
import Screen from '../../components/Screen';
import { supabase } from '../../lib/supabase';
import { TTD } from '../../lib/money';
import type { AppointmentBalance, Payment } from '../../lib/types';

const METHODS: Payment['method'][] = ['cash', 'card', 'transfer', 'online'];

export default function NewPayment() {
  const { appointmentId } = useLocalSearchParams<{ appointmentId?: string }>
();
  const [appt, setAppt] = useState<AppointmentBalance | null>(null);
```

```
  const [amount, setAmount] = useState('');
  const [method, setMethod] = useState<Payment['method']>('cash');
  const [reference, setReference] = useState('');
  const [link, setLink] = useState('');
  const [note, setNote] = useState('');

  useEffect(() => {
    if (!appointmentId) return;
    supabase.from('appointment_balances').select('*').eq('id',
appointmentId).single().then(({ data }) => {
      const a = data as AppointmentBalance; setAppt(a);
      setAmount(String(Math.max(0, Number(a.balance))));
    });
  }, [appointmentId]);

  async function save() {
    const value = Number(amount);
    if (!(value > 0)) return Alert.alert('Enter an amount');
    const online = method === 'online';
    if (online && !link.trim()) return Alert.alert('Paste the payment link');
    const { error } = await supabase.from('payments').insert({
      appointment_id: appt?.id ?? null, client_id: appt?.client_id ?? null,
      amount: value, method,
      status: online ? 'pending' : 'paid',          // online stays pending
until the client pays
      paid_at: online ? null : new Date().toISOString(),
      reference: reference || null, payment_url: online ? link.trim() : null,
note: note || null,
    });
    if (error) return Alert.alert('Save failed', error.message);
    if (online) {
      await Share.share({ message:
        `Hi${appt ? ' ' + appt.client_name.split(' ')[0] : ''}, here's your
payment link for ${TTD(value)}: ${link.trim()}` });
    }
    router.back();
  }

  return (
    <Screen>
      {appt && <Text style={s.sub}>{appt.client_name} · {appt.service_name} ·
balance {TTD(Number(appt.balance))}</Text>}
      <TextInput style={s.big} keyboardType="decimal-pad" value={amount}
```

```
onChangeText={setAmount} placeholder="0.00" />
      <View style={s.row}>
        {METHODS.map((m) => (
          <Pressable key={m} style={[s.chip, method === m && s.on]} onPress=
{() => setMethod(m)}>
            <Text style={method === m ? { color: 'white' } : undefined}>{m}
</Text>
          </Pressable>
        ))}
      </View>
      {method === 'online' ? (
        <TextInput style={s.in} placeholder="Paste payment link"
autoCapitalize="none" value={link} onChangeText={setLink} />
      ) : (
        <TextInput style={s.in} placeholder="Reference (receipt or bank ref,
optional)" value={reference} onChangeText={setReference} />
      )}
      <TextInput style={s.in} placeholder="Note (e.g. tip, product sale)"
value={note} onChangeText={setNote} />
      <Pressable style={s.btn} onPress={save}>
        <Text style={s.btnText}>{method === 'online' ? 'Save and share link'
: 'Record payment'}</Text>
      </Pressable>
    </Screen>
  );
}

const s = StyleSheet.create({
  sub: { color: '#777' },
  big: { fontSize: 36, fontWeight: '700', textAlign: 'center',
backgroundColor: 'white', borderRadius: 12, padding: 16 },
  row: { flexDirection: 'row', gap: 8, flexWrap: 'wrap' },
  chip: { borderWidth: 1, borderColor: '#C2185B', borderRadius: 20,
paddingHorizontal: 14, paddingVertical: 8, backgroundColor: 'white' },
  on: { backgroundColor: '#C2185B' },
  in: { backgroundColor: 'white', borderRadius: 10, padding: 14, borderWidth:
1, borderColor: '#eee' },
  btn: { backgroundColor: '#C2185B', borderRadius: 10, padding: 16,
alignItems: 'center' },
  btnText: { color: 'white', fontWeight: '600' },
});
```

Register the screen in `app/_layout.tsx`, under the appointment lines:

```
<Stack.Screen name="payment/new" options={{ title: 'Payment', presentation:
'modal' }} />
```

`Share.share` opens your phone's share sheet. Pick WhatsApp and the client. Nothing is sent without you choosing.

8.2 Mark a pending online payment as paid

In `app/appointment/[id].tsx` (Part 7.4), add `Share` to the `react-native` import at the top. Then replace the three-line `pays.map(...)` block with this. Pending payments get **Mark paid** and **Resend** buttons:

```
{pays.map((p) => (
  <View key={p.id} style={{ flexDirection: 'row', alignItems: 'center', gap:
8 }}>
    <Text style={[s.sub, { flex: 1 }]}>{p.method} · {TTD(Number(p.amount))} ·
{p.status}</Text>
    {p.status === 'pending' && (
      <>
        <Pressable onPress={async () => {
          await supabase.from('payments').update({ status: 'paid', paid_at:
new Date().toISOString() }).eq('id', p.id);
          load();
        }}><Text style={{ color: '#2E7D32', fontWeight: '600' }}>Mark
paid</Text></Pressable>
        {p.payment_url && (
          <Pressable onPress={() => Share.share({ message: `Your payment
link: ${p.payment_url}` })}>
            <Text style={{ color: '#C2185B' }}>Resend</Text></Pressable>
        )}
      </>
    )}
  </View>
))}
```

Check it works

- [ ] Take a cash payment on an appointment; its balance drops to TT$0.00
- [ ] A partial payment leaves the right balance
- [ ] An online payment shows as **pending** with Mark paid and Resend
- [ ] **Mark paid** changes it to paid, and the balance updates

Part 9: Sales: actual and potential

By the end of this part the Sales tab shows, for any day, week or month: money actually received, money still expected from bookings, a bar for each day split into the two, and breakdowns by payment method and by service.

9.1 The definitions

Figure	Means	Calculated from
Actual sales	Money received	Payments marked **paid**, by the date they were paid
Potential sales	Money still expected	Unpaid balance on **booked** or **completed** appointments, by appointment date
Expected total	What the period should bring in	Actual + potential

Cancelled and no-show appointments are left out of potential sales. Pending online payments count as potential until they're paid.

9.2 Shared sales calculations

The dashboard in Part 10 reuses these, so they live in one file. Weeks start on Monday.

`lib/sales.ts`:

```
import { addDays, addMonths, addWeeks, eachDayOfInterval, format, startOfDay,
startOfMonth, startOfWeek, subDays } from 'date-fns';
import { supabase } from './supabase';
import type { AppointmentBalance } from './types';

export type Period = 'day' | 'week' | 'month';

export function rangeFor(period: Period, anchor: Date) {
  const from = period === 'day' ? startOfDay(anchor)
    : period === 'week' ? startOfWeek(anchor, { weekStartsOn: 1 }) :
startOfMonth(anchor);
  const to = period === 'day' ? addDays(from, 1) : period === 'week' ?
addWeeks(from, 1) : addMonths(from, 1);
  return { from, to };
}
```

```
export function shift(period: Period, anchor: Date, step: number) {
  return period === 'day' ? addDays(anchor, step) : period === 'week' ?
addWeeks(anchor, step) : addMonths(anchor, step);
}

const sum = (xs: number[]) => xs.reduce((a, b) => a + b, 0);

export async function salesBetween(from: Date, to: Date) {
  const [pays, appts] = await Promise.all([
    supabase.from('payments').select('amount, method, paid_at,
appointment_id')
      .eq('status', 'paid').gte('paid_at', from.toISOString()).lt('paid_at',
to.toISOString()),
    supabase.from('appointment_balances').select('*')
      .in('status', ['booked', 'completed'])
      .gte('starts_at', from.toISOString()).lt('starts_at',
to.toISOString()),
  ]);
  const payments = (pays.data ?? []) as { amount: number; method: string;
paid_at: string; appointment_id: string | null }[];
  const bookings = (appts.data ?? []) as AppointmentBalance[];
  const owed = (a: AppointmentBalance) => Math.max(0, Number(a.balance));

  // One entry per day for charts
  const days = eachDayOfInterval({ start: from, end: subDays(to, 1)
}).map((d) => {
    const k = format(d, 'yyyy-MM-dd');
    return {
      date: d,
      actual: sum(payments.filter((p) => format(new Date(p.paid_at), 'yyyy-
MM-dd') === k).map((p) => Number(p.amount))),
      potential: sum(bookings.filter((a) => format(new Date(a.starts_at),
'yyyy-MM-dd') === k).map(owed)),
    };
  });

  const byMethod: Record<string, number> = {};
  for (const p of payments) byMethod[p.method] = (byMethod[p.method] ?? 0) +
Number(p.amount);

  const byService: Record<string, number> = {};
  for (const a of bookings) byService[a.service_name] =
```

```
(byService[a.service_name] ?? 0) + Number(a.price);

  const actual = sum(payments.map((p) => Number(p.amount)));
  const potential = sum(bookings.map(owed));
  return { actual, potential, expected: actual + potential, bookings:
bookings.length, days, byMethod, byService };
}

export type Sales = Awaited<ReturnType<typeof salesBetween>>;
```

9.3 A number tile

`components/StatCard.tsx`:

```
import { Text, View } from 'react-native';

export default function StatCard({ label, value, color = '#222' }: { label:
string; value: string; color?: string }) {
  return (
    <View style={{ flex: 1, minWidth: '45%', backgroundColor: 'white',
borderRadius: 12, padding: 14, gap: 4 }}>
      <Text style={{ color: '#777', fontSize: 13 }}>{label}</Text>
      <Text style={{ fontSize: 20, fontWeight: '700', color }}>{value}</Text>
    </View>
  );
}
```

9.4 The Sales tab

Each bar is one day: green for money received, pink on top for money still expected, so the full height is the day's expected total.

`app/(tabs)/sales.tsx`:

```
import { useCallback, useState } from 'react';
import { Pressable, StyleSheet, Text, View } from 'react-native';
import { useFocusEffect } from 'expo-router';
import { BarChart } from 'react-native-gifted-charts';
import { format } from 'date-fns';
import Screen from '../../components/Screen';
import StatCard from '../../components/StatCard';
import { TTD } from '../../lib/money';
```

```
import { Period, rangeFor, salesBetween, Sales, shift } from
'../../lib/sales';

const GREEN = '#2E7D32', PINK = '#F48FB1';

export default function SalesTab() {
  const [period, setPeriod] = useState<Period>('week');
  const [anchor, setAnchor] = useState(new Date());
  const [data, setData] = useState<Sales | null>(null);
  const { from, to } = rangeFor(period, anchor);

  useFocusEffect(useCallback(() => { salesBetween(from, to).then(setData); },
[period, anchor]));

  const label = period === 'day' ? format(from, 'EEE d MMM yyyy')
    : period === 'week' ? `Week of ${format(from, 'd MMM')}` : format(from,
'MMMM yyyy');

  const bars = (data?.days ?? []).map((d, i, all) => ({
    stacks: [{ value: d.actual, color: GREEN }, { value: d.potential, color:
PINK }],
    // Label every day for a week, every 5th day for a month
    label: all.length <= 7 ? format(d.date, 'EEEEE') : i % 5 === 0 ?
format(d.date, 'd') : '',
  }));

  const top = Object.entries(data?.byService ?? {}).sort((a, b) => b[1] -
a[1]).slice(0, 5);

  return (
    <Screen>
      <View style={s.seg}>
        {(['day', 'week', 'month'] as Period[]).map((p) => (
          <Pressable key={p} style={[s.segBtn, period === p && s.segOn]}
onPress={() => { setPeriod(p); setAnchor(new Date()); }}>
            <Text style={period === p ? { color: 'white', fontWeight: '600' }
: undefined}>{p[0].toUpperCase() + p.slice(1)}</Text>
          </Pressable>
        ))}
      </View>
      <View style={s.nav}>
        <Pressable onPress={() => setAnchor(shift(period, anchor, -1))}><Text
style={s.arrow}>‹</Text></Pressable>
```

```
        <Text style={s.label}>{label}</Text>
        <Pressable onPress={() => setAnchor(shift(period, anchor, 1))}><Text
style={s.arrow}>›</Text></Pressable>
      </View>

      <View style={s.grid}>
        <StatCard label="Actual (received)" value={TTD(data?.actual ?? 0)}
color={GREEN} />
        <StatCard label="Potential (still due)" value={TTD(data?.potential ??
0)} color="#C2185B" />
        <StatCard label="Expected total" value={TTD(data?.expected ?? 0)} />
        <StatCard label="Bookings" value={String(data?.bookings ?? 0)} />
      </View>

      {period !== 'day' && (
        <View style={s.card}>
          <BarChart stackData={bars} barWidth={period === 'week' ? 22 : 6}
spacing={period === 'week' ? 18 : 4}
            noOfSections={4} yAxisThickness={0} xAxisThickness={0} hideRules=
{false} height={180}
            yAxisTextStyle={{ color: '#999', fontSize: 10 }}
xAxisLabelTextStyle={{ color: '#999', fontSize: 10 }} />
          <View style={s.legend}>
            <Text><Text style={{ color: GREEN }}>■</Text> Received   <Text
style={{ color: PINK }}>■</Text> Still due</Text>
          </View>
        </View>
      )}

      <View style={s.card}>
        <Text style={s.h}>Received by method</Text>
        {Object.entries(data?.byMethod ?? {}).map(([m, v]) => (
          <View key={m} style={s.line}><Text>{m}</Text><Text>{TTD(v)}</Text>
</View>
        ))}
      </View>
      <View style={s.card}>
        <Text style={s.h}>Top services (booked value)</Text>
        {top.map(([name, v]) => (
          <View key={name} style={s.line}><Text>{name}</Text><Text>{TTD(v)}
</Text></View>
        ))}
      </View>
```

```
    </Screen>
  );
}

const s = StyleSheet.create({
  seg: { flexDirection: 'row', backgroundColor: 'white', borderRadius: 10,
padding: 4 },
  segBtn: { flex: 1, padding: 10, alignItems: 'center', borderRadius: 8 },
  segOn: { backgroundColor: '#C2185B' },
  nav: { flexDirection: 'row', alignItems: 'center', justifyContent: 'space-
between' },
  arrow: { fontSize: 32, color: '#C2185B', paddingHorizontal: 12 },
  label: { fontSize: 16, fontWeight: '600' },
  grid: { flexDirection: 'row', flexWrap: 'wrap', gap: 8 },
  card: { backgroundColor: 'white', borderRadius: 12, padding: 14, gap: 6 },
  h: { fontWeight: '700', fontSize: 16 },
  line: { flexDirection: 'row', justifyContent: 'space-between' },
  legend: { alignItems: 'center', marginTop: 4 },
});
```

Check it works

Enter test data where you know the answer: two appointments this week at TT$200 and TT$300, with TT$200 paid in cash on the first.

- [ ] Week view: Actual TT$200.00, Potential TT$300.00, Expected TT$500.00
- [ ] The first appointment's bar is all green; the second is all pink
- [ ] Cancelling the second appointment drops Potential to TT$0.00
- [ ] The arrows move between weeks and months correctly

Part 10: The dashboard home screen

By the end of this part, opening the app shows, top to bottom: today's and this week's money, today's bookings, the week's bookings as a chart and a list, client follow-ups due, and six months of revenue.

10.1 Screen layout

Block	Shows	Tap to
Money tiles	Today received, today still due, week received, week expected	Nothing (glance only)
Today	Each booking with time, client, service and paid/owed	Open the appointment
This week	Stacked bar per day (received + still due), then the next 5 upcoming bookings	Open the appointment
Follow-ups	Clients whose follow-up date is overdue or within 7 days	Open the client, or tick **Done**
Last 6 months	Bar chart of money received per month	Nothing

10.2 The code

`app/(tabs)/index.tsx`:

```
import { useCallback, useState } from 'react';
import { Pressable, RefreshControl, ScrollView, StyleSheet, Text, View } from
'react-native';
import { router, useFocusEffect } from 'expo-router';
import { BarChart } from 'react-native-gifted-charts';
import { addDays, addMonths, format, isPast, isToday, startOfMonth, subMonths
} from 'date-fns';
import StatCard from '../../components/StatCard';
import { supabase } from '../../lib/supabase';
import { TTD } from '../../lib/money';
import { rangeFor, salesBetween, Sales } from '../../lib/sales';
import type { AppointmentBalance, Client } from '../../lib/types';

const GREEN = '#2E7D32', PINK = '#F48FB1';

export default function Dashboard() {
  const [today, setToday] = useState<Sales | null>(null);
  const [week, setWeek] = useState<Sales | null>(null);
  const [todayList, setTodayList] = useState<AppointmentBalance[]>([]);
  const [upcoming, setUpcoming] = useState<AppointmentBalance[]>([]);
```

```
  const [followUps, setFollowUps] = useState<Client[]>([]);
  const [months, setMonths] = useState<{ label: string; value: number }[]>
([]);
  const [refreshing, setRefreshing] = useState(false);

  const load = useCallback(async () => {
    const now = new Date();
    const d = rangeFor('day', now), w = rangeFor('week', now);

    const [t, wk, list, next, fu] = await Promise.all([
      salesBetween(d.from, d.to),
      salesBetween(w.from, w.to),
      supabase.from('appointment_balances').select('*').neq('status',
'cancelled')
        .gte('starts_at', d.from.toISOString()).lt('starts_at',
d.to.toISOString()).order('starts_at'),
      supabase.from('appointment_balances').select('*').eq('status',
'booked')
        .gte('starts_at', d.to.toISOString()).lt('starts_at',
w.to.toISOString()).order('starts_at').limit(5),
      supabase.from('clients').select('*').not('follow_up_date', 'is', null)
        .lte('follow_up_date', format(addDays(now, 7), 'yyyy-MM-
dd')).order('follow_up_date'),
    ]);
    setToday(t); setWeek(wk);
    setTodayList((list.data as AppointmentBalance[]) ?? []);
    setUpcoming((next.data as AppointmentBalance[]) ?? []);
    setFollowUps((fu.data as Client[]) ?? []);

    // Six months of received revenue, oldest first
    const starts = Array.from({ length: 6 }, (_, i) =>
startOfMonth(subMonths(now, 5 - i)));
    const totals = await Promise.all(starts.map((m) => salesBetween(m,
addMonths(m, 1))));
    setMonths(starts.map((m, i) => ({ label: format(m, 'MMM'), value:
totals[i].actual })));
  }, []);

  useFocusEffect(useCallback(() => { load(); }, [load]));

  async function clearFollowUp(id: string) {
    await supabase.from('clients').update({ follow_up_date: null,
follow_up_note: null }).eq('id', id);
```

```
    load();
  }

  const weekBars = (week?.days ?? []).map((d) => ({
    label: format(d.date, 'EEEEE'),
    stacks: [{ value: d.actual, color: GREEN }, { value: d.potential, color:
PINK }],
  }));

  return (
    <ScrollView contentContainerStyle={s.page}
      refreshControl={<RefreshControl refreshing={refreshing} onRefresh=
{async () => { setRefreshing(true); await load(); setRefreshing(false); }}
/>}>
      <Text style={s.date}>{format(new Date(), 'EEEE d MMMM')}</Text>

      <View style={s.grid}>
        <StatCard label="Today received" value={TTD(today?.actual ?? 0)}
color={GREEN} />
        <StatCard label="Today still due" value={TTD(today?.potential ?? 0)}
color="#C2185B" />
        <StatCard label="Week received" value={TTD(week?.actual ?? 0)} color=
{GREEN} />
        <StatCard label="Week expected" value={TTD(week?.expected ?? 0)} />
      </View>

      <Text style={s.h}>Today · {todayList.length} booking{todayList.length
=== 1 ? '' : 's'}</Text>
      {todayList.length === 0 && <Text style={s.muted}>Nothing booked today.
</Text>}
      {todayList.map((a) => (
        <Pressable key={a.id} style={s.row} onPress={() =>
router.push(`/appointment/${a.id}`)}>
          <Text style={s.time}>{format(new Date(a.starts_at), 'h:mm a')}
</Text>
          <View style={{ flex: 1 }}>
            <Text style={{ fontWeight: '600' }}>{a.client_name}</Text>
            <Text style={s.muted}>{a.service_name}</Text>
          </View>
          <Text style={{ color: Number(a.balance) > 0 ? '#C2185B' : GREEN,
fontWeight: '600' }}>
            {Number(a.balance) > 0 ? `owes ${TTD(Number(a.balance))}` :
'paid'}
```

```
          </Text>
        </Pressable>
      ))}

      <Text style={s.h}>This week · {week?.bookings ?? 0} bookings</Text>
      <View style={s.card}>
        <BarChart stackData={weekBars} barWidth={22} spacing={18} height=
{150} noOfSections={3}
          yAxisThickness={0} xAxisThickness={0} yAxisTextStyle={{ color:
'#999', fontSize: 10 }}
          xAxisLabelTextStyle={{ color: '#999', fontSize: 10 }} />
        <Text style={[s.muted, { textAlign: 'center' }]}><Text style={{
color: GREEN }}>■</Text> Received   <Text style={{ color: PINK }}>■</Text>
Still due</Text>
      </View>
      {upcoming.map((a) => (
        <Pressable key={a.id} style={s.row} onPress={() =>
router.push(`/appointment/${a.id}`)}>
          <Text style={s.time}>{format(new Date(a.starts_at), 'EEE h:mm a')}
</Text>
          <Text style={{ flex: 1 }}>{a.client_name} · {a.service_name}</Text>
        </Pressable>
      ))}

      <Text style={s.h}>Follow-ups</Text>
      {followUps.length === 0 && <Text style={s.muted}>No follow-ups due this
week.</Text>}
      {followUps.map((c) => {
        const due = new Date(c.follow_up_date + 'T12:00');
        const overdue = isPast(due) && !isToday(due);
        return (
          <View key={c.id} style={s.row}>
            <Pressable style={{ flex: 1 }} onPress={() =>
router.push(`/client/${c.id}`)}>
              <Text style={{ fontWeight: '600' }}>{c.full_name}</Text>
              <Text style={{ color: overdue ? '#B00020' : '#777' }}>
                {overdue ? 'Overdue · ' : ''}{format(due, 'EEE d MMM')}
{c.follow_up_note ? ` · ${c.follow_up_note}` : ''}
              </Text>
            </Pressable>
            <Pressable onPress={() => clearFollowUp(c.id)}><Text style={{
color: GREEN, fontWeight: '600' }}>Done</Text></Pressable>
          </View>
```

```
        );
      })}

      <Text style={s.h}>Received, last 6 months</Text>
      <View style={s.card}>
        <BarChart data={months.map((m) => ({ value: m.value, label: m.label,
frontColor: GREEN }))}
          barWidth={26} spacing={16} height={150} noOfSections={3}
yAxisThickness={0} xAxisThickness={0}
          yAxisTextStyle={{ color: '#999', fontSize: 10 }}
xAxisLabelTextStyle={{ color: '#999', fontSize: 10 }} />
      </View>
    </ScrollView>
  );
}

const s = StyleSheet.create({
  page: { padding: 16, gap: 10, backgroundColor: '#FFF7FA' },
  date: { fontSize: 22, fontWeight: '700' },
  grid: { flexDirection: 'row', flexWrap: 'wrap', gap: 8 },
  h: { fontWeight: '700', fontSize: 17, marginTop: 8 },
  muted: { color: '#777' },
  row: { flexDirection: 'row', alignItems: 'center', gap: 10,
backgroundColor: 'white', padding: 12, borderRadius: 12 },
  time: { width: 80, fontWeight: '600' },
  card: { backgroundColor: 'white', borderRadius: 12, padding: 12, gap: 6 },
});
```

Speed note: the dashboard makes about 18 small database requests when it opens, mostly for the six-month chart. That's fine on the free tier. If it ever feels slow, ask Claude to "move the six-month revenue total into a Postgres function so it's one request."

Check it works

- [ ] Today's tiles match what you entered in Part 9's test
- [ ] Pulling down on the screen refreshes it
- [ ] A client with a follow-up date in 3 days appears; one overdue shows red
- [ ] Tapping **Done** removes the follow-up

Part 11: Follow-ups and reminders

By the end of this part your phone notifies you an hour before each appointment, and each morning at 8:00 with the day's booking count and follow-ups due. Tapping a notification opens the right screen.

These are **local notifications**: your phone schedules them itself from data the app already loaded. No server, no push service and no cost. The trade-off is that the schedule refreshes when you open the app. If you open it at least once a day, it stays current.

11.1 What gets scheduled

Notification	When	Says	Tap opens
Appointment reminder	60 min before each booked appointment in the next 7 days	"Keisha at 2:00 PM · Silk press"	That appointment
Morning brief	8:00 AM each day for the next 7 days, if anything is on	"4 bookings today · 2 follow-ups due"	Home dashboard

iPhones allow at most 64 scheduled notifications per app, so the app only schedules a week ahead and rebuilds the list every time it opens.

11.2 The notifications helper

Replace the placeholder `lib/notifications.ts` from Part 7:

```
import * as Notifications from 'expo-notifications';
import { Platform } from 'react-native';
import { addDays, format, setHours, startOfDay, subMinutes } from 'date-fns';
import { supabase } from './supabase';

const REMIND_BEFORE_MIN = 60;   // change to 30, 120, etc.
const BRIEF_HOUR = 8;           // morning brief time (24-hour clock)

// Show notifications even while the app is open
Notifications.setNotificationHandler({
  handleNotification: async () => ({
    shouldShowBanner: true, shouldShowList: true, shouldPlaySound: true,
shouldSetBadge: false,
  }),
```

```
});

const at = (date: Date): Notifications.NotificationTriggerInput =>
  ({ type: Notifications.SchedulableTriggerInputTypes.DATE, date, channelId:
'reminders' });

export async function setupNotifications() {
  if (Platform.OS === 'android') {
    await Notifications.setNotificationChannelAsync('reminders', {
      name: 'Reminders', importance: Notifications.AndroidImportance.HIGH,
    });
  }
  const { status } = await Notifications.getPermissionsAsync();
  if (status === 'granted') return true;
  return (await Notifications.requestPermissionsAsync()).status ===
'granted';
}

export async function scheduleAppointmentReminder(id: string, client: string,
service: string, start: Date) {
  const when = subMinutes(start, REMIND_BEFORE_MIN);
  if (when <= new Date() || when > addDays(new Date(), 7)) return;  //
syncReminders picks up later ones
  await Notifications.scheduleNotificationAsync({
    identifier: `appt-${id}`,
    content: { title: `${client} at ${format(start, 'h:mm a')}`, body:
service, data: { url: `/appointment/${id}` } },
    trigger: at(when),
  });
}

export async function cancelAppointmentReminder(id: string) {
  await Notifications.cancelScheduledNotificationAsync(`appt-${id}`).catch(()
=> {});
}

// Rebuild everything for the next 7 days. Called when the app opens or comes
back to the front.
export async function syncReminders() {
  await Notifications.cancelAllScheduledNotificationsAsync();
  const now = new Date();
  const end = addDays(startOfDay(now), 7);
```

```
  const [{ data: appts }, { data: clients }] = await Promise.all([
    supabase.from('appointment_balances').select('id, client_name,
service_name, starts_at')
      .eq('status', 'booked').gte('starts_at',
now.toISOString()).lt('starts_at', end.toISOString()),
    supabase.from('clients').select('follow_up_date').not('follow_up_date',
'is', null)
      .lte('follow_up_date', format(end, 'yyyy-MM-dd')),
  ]);

  for (const a of appts ?? []) {
    await scheduleAppointmentReminder(a.id, a.client_name, a.service_name,
new Date(a.starts_at));
  }

  for (let i = 0; i < 7; i++) {
    const day = addDays(startOfDay(now), i);
    const when = setHours(day, BRIEF_HOUR);
    if (when <= now) continue;
    const key = format(day, 'yyyy-MM-dd');
    const bookings = (appts ?? []).filter((a) => format(new
Date(a.starts_at), 'yyyy-MM-dd') === key).length;
    const due = (clients ?? []).filter((c) => c.follow_up_date! <=
key).length;  // includes overdue
    if (!bookings && !due) continue;
    const parts = [
      bookings && `${bookings} booking${bookings > 1 ? 's' : ''} today`,
      due && `${due} follow-up${due > 1 ? 's' : ''} due`,
    ].filter(Boolean);
    await Notifications.scheduleNotificationAsync({
      identifier: `brief-${key}`,
      content: { title: 'Good morning', body: parts.join(' · '), data: { url:
'/' } },
      trigger: at(when),
    });
  }
}
```

11.3 Wire it into the root layout

In `app/_layout.tsx` (Part 4.2), add these imports:

```
import { AppState } from 'react-native';
import { router } from 'expo-router';
import * as Notifications from 'expo-notifications';
import { setupNotifications, syncReminders } from '../lib/notifications';
```

Then add this second `useEffect` inside `RootLayout`, under the first one:

```
useEffect(() => {
  if (!session) return;
  setupNotifications().then((ok) => ok && syncReminders());

  // Re-sync whenever the app comes back to the front
  const appSub = AppState.addEventListener('change', (st) => { if (st ===
'active') syncReminders(); });

  // Tapping a notification opens its screen
  const open = (r: Notifications.NotificationResponse | null) => {
    const url = r?.notification.request.content.data?.url as string |
undefined;
    if (url) router.push(url as any);
  };
  Notifications.getLastNotificationResponseAsync().then(open);   // app was
closed
  const tapSub = Notifications.addNotificationResponseReceivedListener(open);
// app was open

  return () => { appSub.remove(); tapSub.remove(); };
}, [session]);
```

11.4 Testing notifications

Local notifications should work in Expo Go. If they don't appear on Android, Expo Go is the limit, not your code. Make a **development build** (Part 13.2), which behaves like the real app.

To test quickly, temporarily set `REMIND_BEFORE_MIN = 1` and book an appointment 3 minutes from now. Lock your phone and wait. Set it back to 60 afterwards.

Check it works

- [ ] The app asks for notification permission after you log in
- [ ] A test appointment reminder arrives and tapping it opens that appointment

- [ ] Cancelling an appointment stops its reminder
- [ ] The next morning's brief arrives at 8:00 with the right counts

Part 12: Testing and backups

By the end of this part you'll have automated tests for the money and date maths, a full test run on both phone types, a week of real use, and a monthly backup routine.

12.1 Automated tests for the maths

The sales numbers are the part most likely to be quietly wrong, so test the date ranges they depend on.

```
$ npx expo install jest-expo jest @types/jest -- --save-dev
```

In `package.json`, add to `"scripts"`: `"test": "jest"`, and add a new top-level section: `"jest": { "preset": "jest-expo" }`.

`lib/__tests__/sales.test.ts`:

```
import { rangeFor, shift } from '../sales';
import { TTD } from '../money';

// lib/sales imports supabase; replace it with a dummy for these tests
jest.mock('../supabase', () => ({ supabase: {} }));

test('week starts on Monday', () => {
  const { from, to } = rangeFor('week', new Date(2026, 9, 15)); // Thu 15 Oct
2026
  expect(from.getDay()).toBe(1);           // Monday
  expect(from.getDate()).toBe(12);
  expect(to.getDate()).toBe(19);           // the following Monday
});

test('month range covers the whole month', () => {
  const { from, to } = rangeFor('month', new Date(2026, 1, 10)); // Feb 2026
  expect(from.getDate()).toBe(1);
  expect(to.getMonth()).toBe(2);           // 1 March
});

test('shift moves by one period', () => {
  expect(shift('month', new Date(2026, 0, 31), 1).getMonth()).toBe(1); // Jan
```

```
31 -> Feb
});

test('formats TT dollars', () => {
  expect(TTD(1234.5)).toBe('TT$1,234.50');
});
```

```
$ npm test
```

12.2 Full manual test run

Run this on **both** an Android phone and an iPhone (borrow one if needed; Expo Go works on any phone signed in to your Expo account).

- [ ] **Login:** wrong password fails; right one works; reopen stays logged in; log out works
- [ ] **Passcode:** create, wrong code rejected, 5 wrong tries locks for a minute, forgot-passcode reset works
- [ ] **Services:** add, edit price, retire; retired service doesn't appear when booking
- [ ] **Clients:** add with every field filled; add with only a name; search; edit; delete
- [ ] **Booking:** book, overlap warning, reschedule, done, no-show, cancel, undo
- [ ] **Payments:** full cash, partial card, transfer with reference, online link shared to WhatsApp, mark paid
- [ ] **Sales:** day/week/month figures match a hand calculation; arrows move periods
- [ ] **Dashboard:** tiles, today list, week chart, follow-ups, 6-month chart all correct
- [ ] **Reminders:** appointment reminder and morning brief arrive; tapping opens the right screen
- [ ] **No internet:** turn on airplane mode and open each tab. The app should show empty lists or an error, not crash
- [ ] **Big text:** turn up the phone's font size; nothing important is cut off

12.3 A real week alongside your current system

For one week, record everything in the app **and** in whatever you use now (book, spreadsheet, WhatsApp). At the end of the week, compare totals. Any difference is a bug or a habit to fix before you rely on the app alone.

12.4 Backups

Supabase's free plan doesn't make automatic backups. On the first of each month:

1. In Supabase, open **Table Editor**.
2. For each of services, clients, appointments and payments: open the table, then **Export → Export table as CSV**.
3. Save the four files in a dated folder on Google Drive, e.g. `Salon backups/2026-11`.

Set a monthly reminder in your phone's calendar so you don't forget.

12.5 Error checks worth adding

Most screens above `Alert` the error when a save fails, but the list screens ignore load errors silently. Once everything works, ask Claude: "Add error handling to every Supabase load in my app: show a retry message when a query fails."

Part 13: Build and install the real app

By the end of this part the app is installed on your phone as a normal app with its own icon, without Expo Go.

Because only you use it, you probably **don't need the public app stores**:

Your phone	Cheapest route	Cost
Android	Build an APK file and install it directly	Free
iPhone	Build with an Apple Developer account and install through TestFlight	US$99/year
Both, publicly listed	Google Play + App Store	US$25 once + US$99/year, plus review hurdles

Publishing publicly only makes sense if you plan to sell the app to other salons later. Section 13.6 covers it.

13.1 Name, icon and IDs

Edit `app.json`:

```
{
  "expo": {
    "name": "Salon Dashboard",
    "slug": "salon-dashboard",
    "version": "1.0.0",
    "icon": "./assets/images/icon.png",
    "ios": { "bundleIdentifier": "com.yoursalonname.dashboard" },
    "android": {
      "package": "com.yoursalonname.dashboard",
      "adaptiveIcon": { "foregroundImage": "./assets/images/adaptive-
icon.png", "backgroundColor": "#FFF7FA" }
    },
    "plugins": ["expo-router", "expo-secure-store", "expo-notifications"]
  }
}
```

Keep any other settings the template already had. Use your salon's name in the IDs: lowercase, no spaces. **You can't change the IDs after publishing.** The icon must be a 1024×1024 PNG; you can make one in Canva.

13.2 Set up EAS

```
$ npm install -g eas-cli
$ eas login
$ eas build:configure
```

Your `.env` file isn't uploaded to the build servers because it's in `.gitignore`. Give EAS the two values instead:

```
$ eas env:create --name EXPO_PUBLIC_SUPABASE_URL --value https://YOUR-
PROJECT.supabase.co --environment production --environment preview --
environment development --visibility plaintext
$ eas env:create --name EXPO_PUBLIC_SUPABASE_KEY --value YOUR-PUBLISHABLE-KEY
--environment production --environment preview --environment development --
visibility plaintext
```

Edit `eas.json` so each profile uses those values, and so the `preview` Android build makes an installable APK:

```
{
  "build": {
    "development": { "developmentClient": true, "distribution": "internal",
"environment": "development" },
    "preview": { "distribution": "internal", "environment": "preview",
"android": { "buildType": "apk" } },
    "production": { "environment": "production", "autoIncrement": true }
  }
}
```

Optional development build: if something didn't work in Expo Go (often notifications on Android), run `npx expo install expo-dev-client`, then `eas build --profile development --platform android`. Install the result and start the code with `npx expo start --dev-client --tunnel`.

13.3 Android: install the APK (free)

```
$ eas build --profile preview --platform android
```

The build runs on Expo's servers. Free builds can wait in a queue, so allow 15–60 minutes. When it finishes, the Terminal shows a link and QR code.

1. Open the link on your Android phone and download the APK.
2. When Android warns about installing from an unknown source, allow it for your browser.
3. Install, open, and log in.

13.4 iPhone: TestFlight (US$99/year)

1. Join the Apple Developer Program as an individual. Approval can take a day or two.
2. Build and send it to Apple:

```
$ eas build --profile production --platform ios
$ eas submit --platform ios
```

EAS asks for your Apple ID and sets up certificates for you. No Mac needed.

3. In App Store Connect, open the app, go to **TestFlight**, and add yourself under **Internal Testing**. Internal testers don't need Apple review.
4. Install Apple's **TestFlight** app on your iPhone and install your app from it.

TestFlight builds expire after 90 days, so make a fresh build at least every three months.

13.5 Fixing bugs without rebuilding

Small JavaScript changes can go straight to your installed app with **EAS Update**, which is free for up to 1,000 users a month:

```
$ npx expo install expo-updates
$ eas update:configure
$ eas update --channel preview --message "Fix follow-up date"
```

Rebuild after an `eas update:configure` once; after that, `eas update` sends fixes in minutes. You need a full rebuild only when you add a library with native code (anything installed with `npx expo install` that has a `plugins` entry or that the docs say needs a rebuild).

13.6 Public store listing (only if you'll offer it to others)

- **Google Play:** a new personal developer account must run a closed test with at least 12 testers for 14 days before it can publish publicly. A business account (with a D-U-N-S number) avoids this.
- **App Store:** reviewers need to log in, so create a demo account with sample data and give its details in the review notes. You also need a privacy policy web page, because the app stores client contact details.
- Both stores ask what data you collect. Here that's client names, phone numbers and emails, stored in Supabase and not shared with anyone else.

```
$ eas build --profile production --platform android
$ eas submit --platform android
```

Check it works

- [ ] The app opens from its own icon with Expo Go closed
- [ ] Login, data and notifications all work in the installed version
- [ ] An `eas update` change appears after closing and reopening the app twice

Using Claude alongside this tutorial

Because this tutorial already contains most of the code, Claude's job shifts from writing everything to fixing errors, adjusting features and explaining things, which uses far fewer tokens than building from scratch. The schedule below maps each part to a 5-hour Claude usage window. Token figures are rough estimates of my own, not official numbers.

Slot	Parts	Main Claude use	Est. tokens
1	Part 1 + Part 2	Chromebook setup errors, first run	50K–150K
2	Part 3 + Part 4	SQL errors, login issues	100K–250K
3	Part 5 + Part 6	Service menu and client screens	150K–300K
4	Part 7	Calendar and booking	150K–350K
5	Part 8	Payment form, mark paid	100K–250K
6	Part 9 + Part 10	Sales maths, charts	150K–350K
7	Part 11	Notifications	100K–250K
8	Part 12	Testing, fixing bugs found	150K–400K
9	Part 13	Build errors, TestFlight	50K–200K
	Total		**About 1–2.5M tokens**

Give Claude your project's context once

If you use Claude Code, create `CLAUDE.md` in your project folder. Claude reads it at the start of every session, so you never have to re-explain the app. In a regular chat, paste it at the top or keep it in a Project's instructions.

```
# Salon Dashboard

Private salon management app for one owner (me). Trinidad & Tobago; currency
TT$; weeks start Monday.

## Stack
Expo (React Native, TypeScript), Expo Router, Supabase (Postgres + auth),
react-native-calendars, react-native-gifted-charts, expo-notifications (local
only), date-fns.
I develop on a Chromebook (Linux) and run with `npx expo start --tunnel`.

## Data
Tables: services, clients, appointments, payments. View: appointment_balances
(paid, balance).
All tables have RLS: owner_id = auth.uid().
Actual sales = paid payments by paid_at. Potential = unpaid balance on
booked/completed appointments.

## Rules
- I know basic coding; explain changes briefly.
- Change only the files needed; show the full updated function, not the whole
file.
- Never put secret keys in the app.
```

Prompts that save tokens

- **Paste the exact error and the file it names:** "Here's the red error screen text and `app/(tabs)/sales.tsx`. What's wrong?" One file beats the whole project.
- **Ask for one change at a time:** "Add a 'Deposit' payment method" rather than "improve payments".
- **Start a fresh conversation for each part.** Long conversations make every message more expensive.
- **Ask before big changes:** "Explain how you'd add product sales tracking before writing code."

Ideas for later versions

Once the core app is running, these make good one-slot projects with Claude:

- Track retail products and stock
- Client birthday reminders in the morning brief

- Export a month's sales to PDF for your accountant
- A deposit required when booking certain services

Troubleshooting

Problem	Likely cause	Fix
Phone can't connect to Expo; QR scan hangs	Chromebook's Linux network is separate from your Wi-Fi	Always use `npx expo start --tunnel`
`nvm: command not found`	Terminal hasn't reloaded	Close and reopen the Terminal
Red screen: "supabaseUrl is required"	`.env` missing or Expo not restarted	Check `.env` names match exactly, then stop and restart Expo
Lists are always empty, no error	RLS blocking: rows have no owner, or you're not logged in	Add rows through the app, not the SQL editor; check you're logged in
"new row violates row-level security policy"	Inserting while logged out	Log in again; check the session in `_layout.tsx`
Sales totals look a day off	Dates compared in UTC instead of local time	Use the `rangeFor` helper; never build date ranges from `YYYY-MM-DD` strings alone
"Unable to resolve module…"	Library not installed	`npx expo install <name>`, then restart Expo with `npx expo start --tunnel -c`
Charts don't show	Missing chart dependencies	`npx expo install react-native-svg expo-linear-gradient`
Notifications never arrive	Permission denied, or Expo Go limitation on Android	Check phone Settings → Apps → notifications; try a development build (13.2)

Problem	Likely cause	Fix
App says "network request failed" after a week away	Supabase free project was paused	Open the Supabase dashboard and restore the project
EAS build fails on missing Supabase URL	Env vars not set for that profile	Re-run the `eas env:create` commands with all three `--environment` flags
`eas build` waits a long time	Free-tier build queue	Normal; build less often, or start it before a break

When you're stuck for more than 20 minutes: copy the full error text, note the file and what you last changed, and give all three to Claude. To undo a bad change, `git status` shows what changed and `git checkout -- <file>` restores the last committed version, so commit after every working step.

Sources

- Supabase pricing and free project pausing
- Expo EAS pricing and EAS plans